

1 函数增长率排序

用下面的函数表，按照增长率上升的顺序排列它们。即，如果在你的表中，函数 $g(n)$ 紧跟在 $f(n)$ 的后面，那么应该满足 $f(n)$ 是 $O(g(n))$ 。

$$g_1(n) = 2^{\sqrt{\log n}}$$

$$g_2(n) = 2^n$$

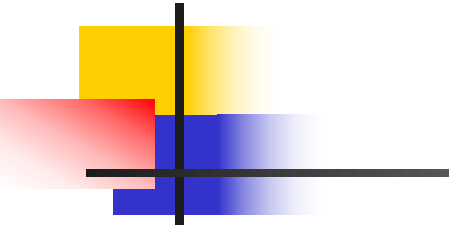
$$g_3(n) = n^{4/3}$$

$$g_4(n) = n(\log n)^3$$

$$g_5(n) = n^{\log n}$$

$$g_6(n) = 2^{2^n}$$

$$g_7(n) = 2^{n^2}$$



接下来比较 g_7 和 g_6 。 g_7 是 $2^{\{n^2\}}$ ，而 g_6 是 $2^{\{2^n\}}$ 。当 n 增大时， n^2 的增长速度远慢于 2^n 。比如， $n=10$ ， $n^2=100$ ， $2^{10}=1024$ ；当 $n=20$ ， $n^2=400$ ，而 $2^{20}=1,048,576$ 。因此，指数部分 2^n 比 n^2 增长快得多，所以 $2^{\{2^n\}}$ 比 $2^{\{n^2\}}$ 增长快得多。因此， $g_6(n) > g_7(n)$ 。所以顺序最后两位是 $g_7 < g_6$ 。

现在确认各个函数之间的比较是否正确：

例如， $g_4(n) = n (\log n)^3$ 的增长率比 $g_3(n)=n^{\{4/3\}}$ 慢。对吗？

是的，如前所述，当 n 足够大时， $n^{\{4/3\}}$ 比 $n (\log n)^3$ 增长快。

那现在顺序是否正确？

目前顺序：

1. $g_1: 2^{\sqrt{\log n}}$
2. $g_4: n (\log n)^3$
3. $g_3: n^{\{4/3\}}$
4. $g_5: n^{\{\log n\}}$
5. $g_2: 2^n$
6. $g_7: 2^{\{n^2\}}$
7. $g_6: 2^{\{2^n\}}$

现在需要确认是否还有其他错误。例如， g_5 的增长率是否确实在 g_2 之前？



2 玻璃瓶样品强度测试

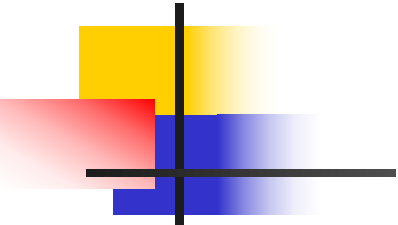
你正在对不同的玻璃瓶样品做强度测试以确定它们从多高的高度掉下来而仍旧不碎。对某类特定的瓶子设计这样一个实验。你有一个具有 n 个横挡的阶梯，并且你想找出最高的横挡，能使一个瓶子的样品从那里下落而不被摔碎。我们把它称为“最高的安全横挡”。

尝试二分搜索可能是一种自然的想法：使一个瓶子从中间的横挡下落，看看它是否摔碎，然后依赖于这个结果递归地从 $n/4$ 横挡或者 $3n/4$ 的横挡进行尝试。但是这个办法有缺点，在找到答案之前，你可能摔碎了一大堆瓶子。

如果你的主要目标是保护瓶子，另一方面，你可以尝试下面的策略。从第一个横挡开始让瓶子下落，然后第二个横挡，继续下去，每次向上爬一个高度直到瓶子摔碎为止。以这种方式，你只需要一个瓶子——在它摔碎的时刻，你得到正确的答案——但是你可能不得不把它下落 n 次（而不是在二分搜索求解时的 $\log n$ 次）。

于是，这里是某种权衡：似乎是如果你愿意打碎更多的瓶子，你就可以执行更少的下落。为了更好地理解这种权衡在量级上是怎样起作用的，让我们考虑给定固定 $k \geq 1$ 个瓶子的“预算”。怎样来运行这个实验。换句话说，你必须确定正确的答案——最高的安全横挡——并且在这个过程中至多可以用 k 个瓶子。

- (a) 假设只给你 $k = 2$ 个瓶子。描述一种找到最高安全横挡的策略，对某个比线性函数增长更慢的函数 $f(n)$ ，要求一个瓶子至多落 $f(n)$ 次（换句话说，应该满足 $\lim_{n \rightarrow \infty} f(n)/n = 0$ ）。
- (b) 假设你的瓶子数的预算是个给定的 $k > 2$ 。描述一种至多使用 k 个瓶子找到最高安全横挡的策略。如果 $f_k(n)$ 表示依照你的策略需要下落瓶子的次数，那么函数 $f_1, f_2, f_3, \dots$ 应该有这样的性质，每个函数渐近增长比前面的函数要慢：对每个 k , $\lim_{n \rightarrow \infty} f_k(n)/f_{k-1}(n) = 0$ 。



2. 玻璃瓶样品强度测试

(a) $k = 2$ 个瓶子的策略

1. 分块策略：将阶梯分为 $\sqrt{n}$ 个块，每个块大小为 $\sqrt{n}$ 。
2. 第一瓶测试：依次测试每个块的最后一个横档（如 $\sqrt{n}, 2\sqrt{n}, \dots$ ），直到瓶子破碎。
3. 第二瓶测试：在破碎块内，从低到高逐个测试横档。
4. 总次数：最多 $2\sqrt{n}$ ，满足 $\lim_{n \rightarrow \infty} \frac{2\sqrt{n}}{n} = 0$ 。

(b) $k > 2$ 个瓶子的策略

1. 递归分块：将阶梯分为 $n^{1/k}$ 个块，每个块大小为 $n^{1-1/k}$ 。
2. 逐层细化：用第一个瓶子确定破碎块后，剩余 $k - 1$ 个瓶子递归处理子块。
3. 总次数：最多 $k \cdot n^{1/k}$ ，满足 $\lim_{n \rightarrow \infty} \frac{f_k(n)}{f_{k-1}(n)} = 0$ 。
(例如， $k = 3$ 时，次数为 $O(n^{1/3})$ ，比 $O(n^{1/2})$ 更慢)

答案

1. 函数排序结果：

$$g_1 \prec g_4 \prec g_3 \prec g_5 \prec g_2 \prec g_7 \prec g_6$$

2. (a) 策略：分块大小为 $\sqrt{n}$ ，最多 $2\sqrt{n}$ 次测试。

(b) 策略：递归分块 $n^{1/k}$ ，次数为 $O(k \cdot n^{1/k})$ 。

2 玻璃瓶样品强度测试

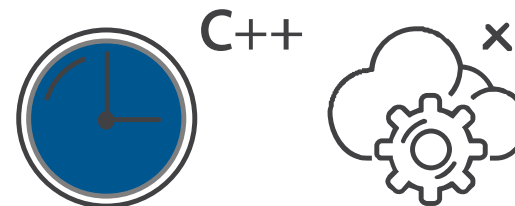
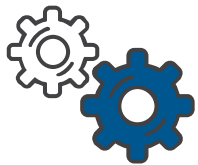
- (a) 为了简化问题, 假设 n 是一个完全平方数。我们从 $\sqrt{n}$ 的倍数高度 (即从 $\sqrt{n}, 2\sqrt{n}, 3\sqrt{n}, \dots$) 开始往下扔第一个罐子, 直到它破碎为止。

如果从最高层往下扔并且它幸存了, 那么我们的工作也就完成了。否则, 假设它在高度 $j\sqrt{n}$ 处破碎。那么我们知道最高的安全层位于 $(j-1)\sqrt{n}$ 和 $j\sqrt{n}$ 之间, 因此我们从 $1 + (j-1)\sqrt{n}$ 层开始往上扔第二个罐子, 每次增加一层。这样, 我们每个罐子最多扔 $\sqrt{n}$ 次, 总共最多扔 $2\sqrt{n}$ 次。

如果 n 不是一个完全平方数, 那么我们从 $\lfloor \sqrt{n} \rfloor$ 的倍数高度扔第一个罐子, 然后对第二个罐子应用上述规则。这样, 我们最多扔第一个罐子 $2\sqrt{n}$ 次, 第二个罐子最多扔 $\sqrt{n}$ 次, 仍然得到 $O(\sqrt{n})$ 的界限。

- (b) 我们通过归纳法声称 $f_k(n) \leq 2kn^{1/k}$ 。我们首先从 $\lfloor n^{(k-1)/k} \rfloor$ 的倍数高度扔第一个罐子。这样, 我们最多扔第一个罐子 $2n/n^{(k-1)/k} = 2n^{1/k}$ 次, 从而将可能的层数缩小到最多 $n^{(k-1)/k}$ 的长度区间。

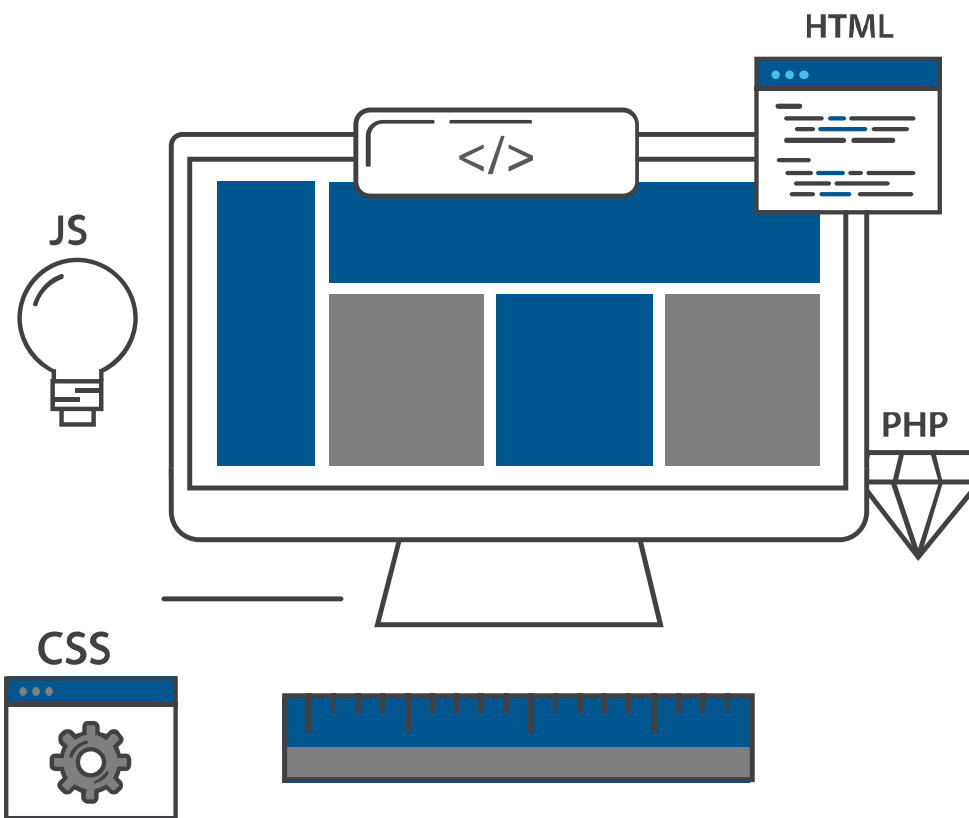
然后我们递归地对 $k-1$ 个罐子应用这个策略。通过归纳, 它最多使用 $2(k-1)(n^{(k-1)/k})^{1/(k-1)} = 2(k-1)n^{1/k}$ 次。加上使用第一个罐子所做的 $\leq 2n^{1/k}$ 次, 我们得到了 $2kn^{1/k}$ 的界限, 完成了归纳步骤。



第四章 贪心算法

李翔

<https://implus.github.io/>





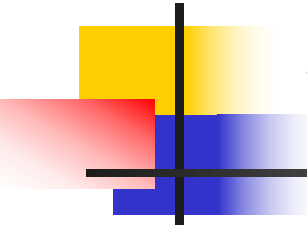
内容安排

- 4.1 区间调度：贪心算法领先
- 4.2 最小延迟调度：一个交换论证
- 4.3 最优高速缓存：一个更复杂的交换论证
- 4.4 一个图的最短路径
- 4.5 最小生成树问题
- 4.6 实现Kruskal 算法
- 4.7 聚类
- 4.8 Huffman 码与数据压缩



贪心算法

- 直观上说，一个算法是贪心的，如果此算法是通过一些小的步骤来建立一个解，在每一步根据**局部情况选择**一个决定使得某些主要的指标能得到优化。
- 当一个贪心算法成功求解了一个非平凡的问题，通常隐含了某些有趣的，与问题本身结构有关的一些性质：**存在一个局部判断规则**可以用来构造问题的**最优解**。



贪心算法

- 本章逐渐介绍两个基本的方法来证明一个贪心算法对一个问题能够提供一个最优解。
 - ✓ 1. 贪心算法领先的概念： 每一步都比其他的算法好，从而证明产生了一个最优解。
 - ✓ 2. 交换论证： 考虑对这个问题的任何可能解，逐渐把它转换成由贪心算法找到的解，而且不影响解的质量。从而证明了贪心法找到了一个至少与其它解一样好的解。

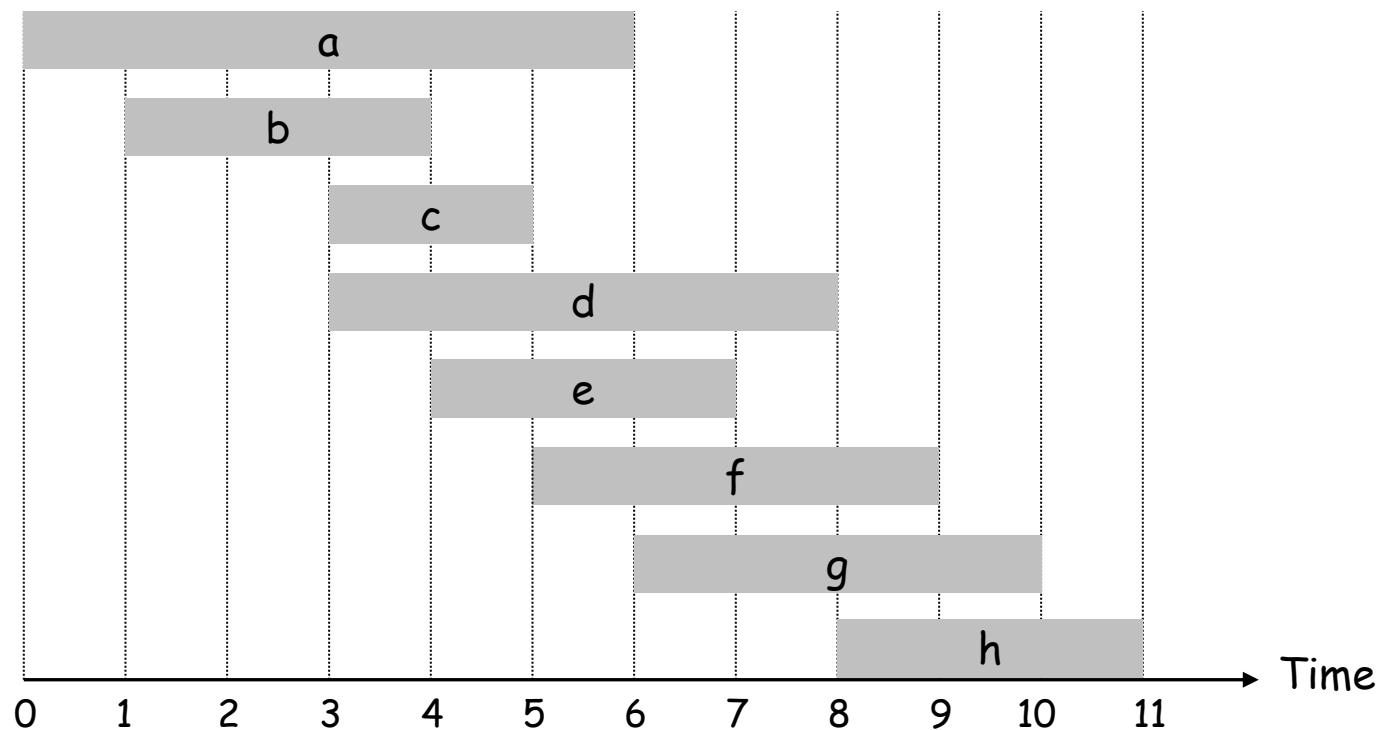


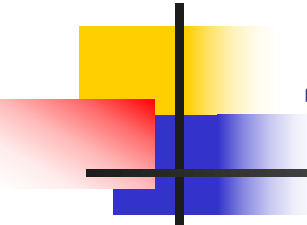
4.1 区间调度：贪心算法领先

- 回顾第一章考虑的五个典型问题的第一个问题：一组需求 $\{1, 2, \dots, n\}$ ；第 i 个需求与一个始于 $s(i)$ 且止于 $f(i)$ 的时间区间相对应。如果没有两个需求在时间上重叠，我们就说需求的子集是相容的。目标是给出一个最大的相容子集。

4.1 区间调度

- 区间调度问题:
- 任务 (需求) j 开始时间 s_j , 结束时间 f_j .
- 两个任务是相容的如果任务的时间区间不相交
- 目的: 找到数目尽可能多的相容任务





4.1 区间调度

- 那么最好的方案应该是什么？
- 吸取前面的经验---与开始时间、区间的宽度、最少“冲突”都有一点联系
- 应该选择什么样的规则？
- 我们应该接受**最早结束**的需求，即 **$f(i)$ 尽可能小**的需求*i*为第一个需求。这样的好处是**资源尽可能早被释放**，以便于安排下面的需求。



贪心算法

- 初始令 R 是所有需求的集合, 设 A 为空
- While R 非空
 - 选择一个最小结束时间的需求 $i \in R$
 - 把 i 加到 A 中
 - 从 R 中删除与需求 i 不相容的所有需求
- Endwhile
- 返回集合 A 作为被接受的需求集合

DeepSeek-R1

区间调度贪心算法正确性证明

区间调度：贪心算法的简易证明方法？



已深度思考（用时 144 秒）

区间调度贪心算法正确性证明

最早的区间)，然后通过数学归纳法或者逐步替换的方法，证明整个贪心解的结构与某个最优解一致，从而其数目相同。

区间调度问题中，贪心算法选择结束时间最早的区间，并按此策略依次选取不重叠的区间。以下是该算法正确性的简易证明：

证明步骤：

- 排序与贪心选择：**将区间按结束时间升序排列。设贪心算法依次选出的区间为 $g_1, g_2, \dots, g_k$ ，其中 g_1 是结束最早的区间。
- 存在性证明：**假设存在一个最优解 $O = \{o_1, o_2, \dots, o_m\}$ 。若 O 不包含 g_1 ，则用 g_1 替换 o_1 。由于 g_1 的结束时间更早，替换后的区间集 $O' = \{g_1, o_2, \dots, o_m\}$ 仍为有效解（不重叠），且数目不变，说明存在包含 g_1 的最优解。
- 归纳递推：**在选定 g_1 后，排除所有与 g_1 重叠的区间，剩余区间构成子问题。根据归纳假设，贪心算法在子问题中仍能选出最优解 $\{g_2, \dots, g_k\}$ 。因此，总解 $\{g_1, g_2, \dots, g_k\}$ 为全局最优。
- 替换法验证：**若贪心解数目 k 小于最优解数目 m ，则存在某一步贪心选择 g_i 后，最优解中后续可选的区间数更多。但由于 g_i 结束时间不晚于最优解对应区间的结束时间，后续可选区间数不可能更多，故 $k \geq m$ 。结合最优解定义 $m \geq k$ ，得 $k = m$ 。

结论：贪心算法通过每次选择结束最早的区间，确保每一步均为局部最优，最终得到全局最优解。

text

复制

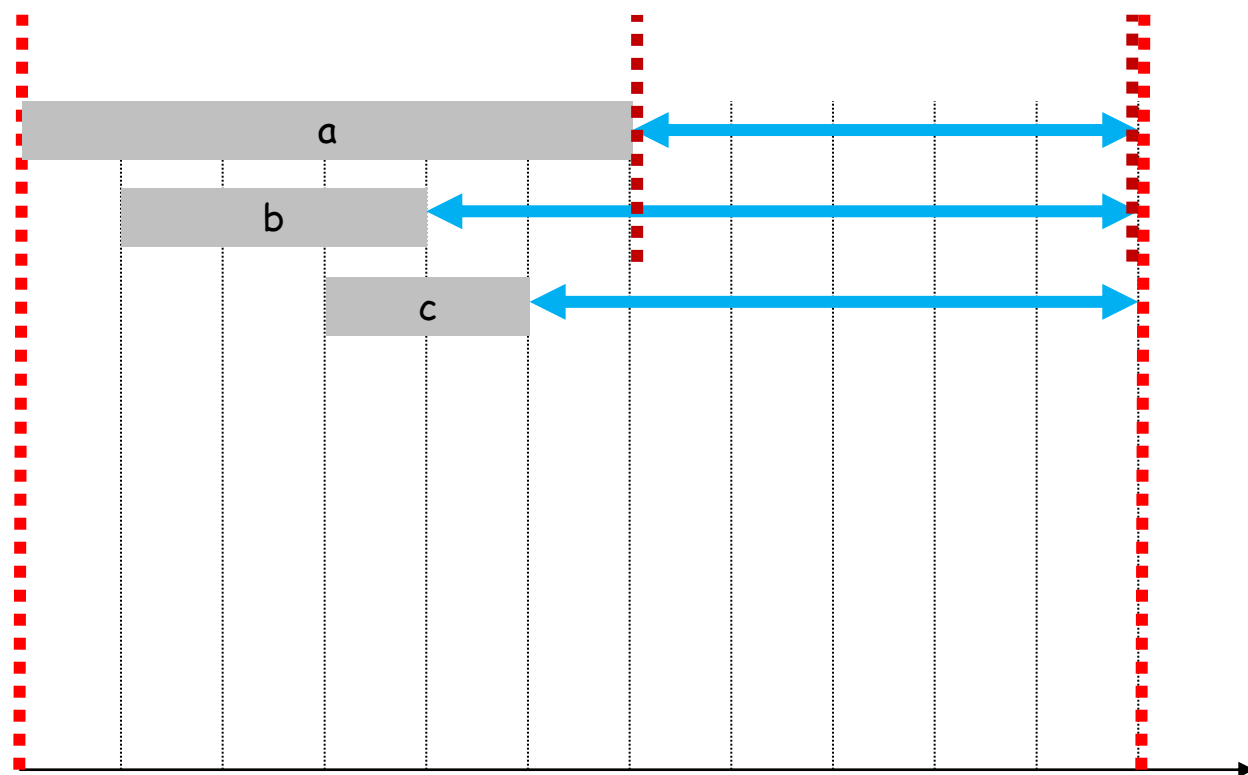
区间调度的贪心算法证明步骤如下：

- **排序**：**将区间按结束时间从小到大排序。
- **贪心选择**：**每次选取结束最早且与前一个选中区间不重叠的区间。
- **存在性**：**任何最优解均可通过替换第一个区间为贪心选择的区间，转化为包含贪心选择的最优解。
- **归纳递推**：**剩余区间构成子问题，递归应用贪心策略，确保每一步选择均最优。

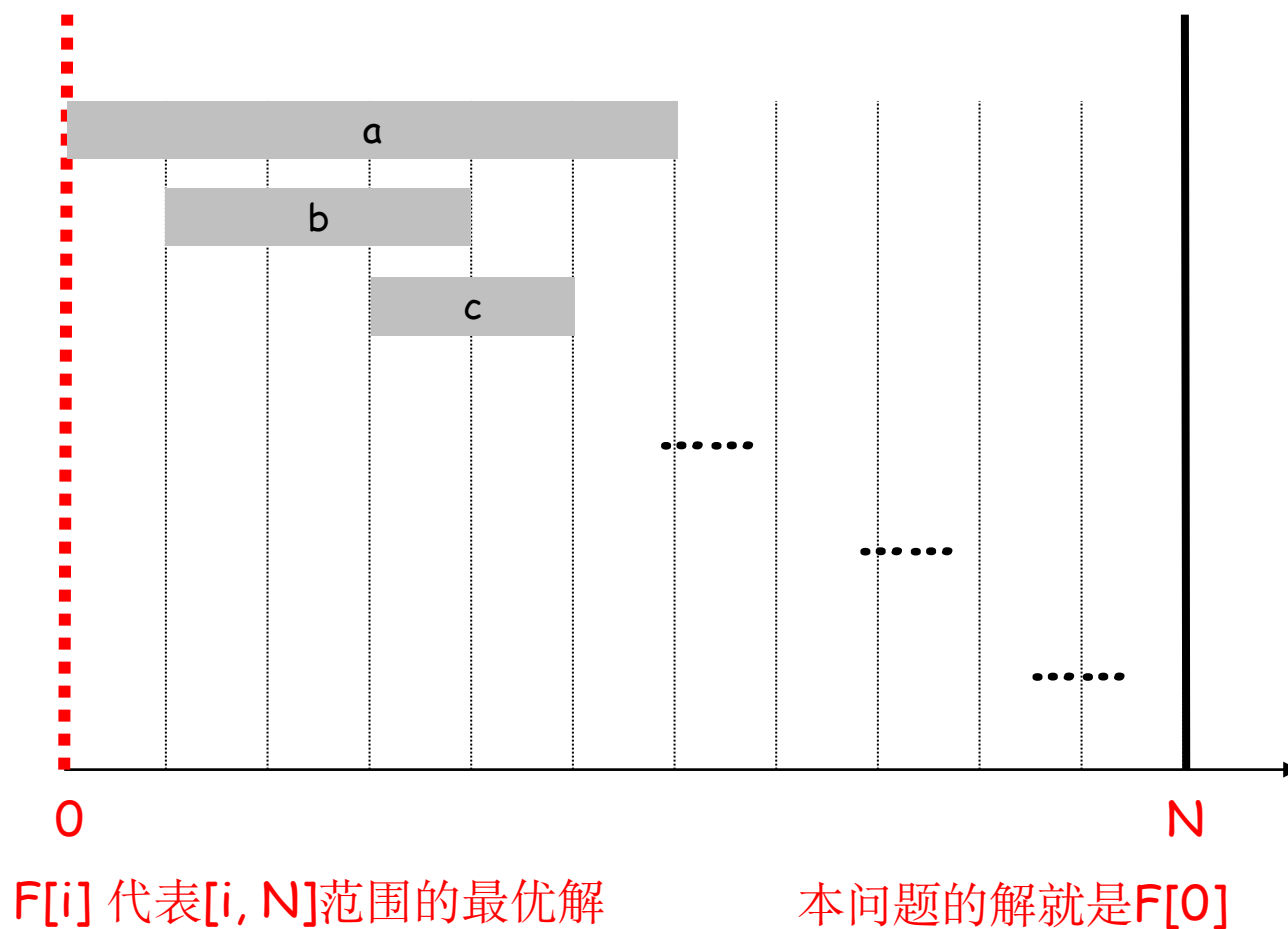
综上，贪心算法选出的区间数等于最大可能的不重叠区间数，证明其正确性。

全新的思路 and 简单理念

递归：问题规模缩小

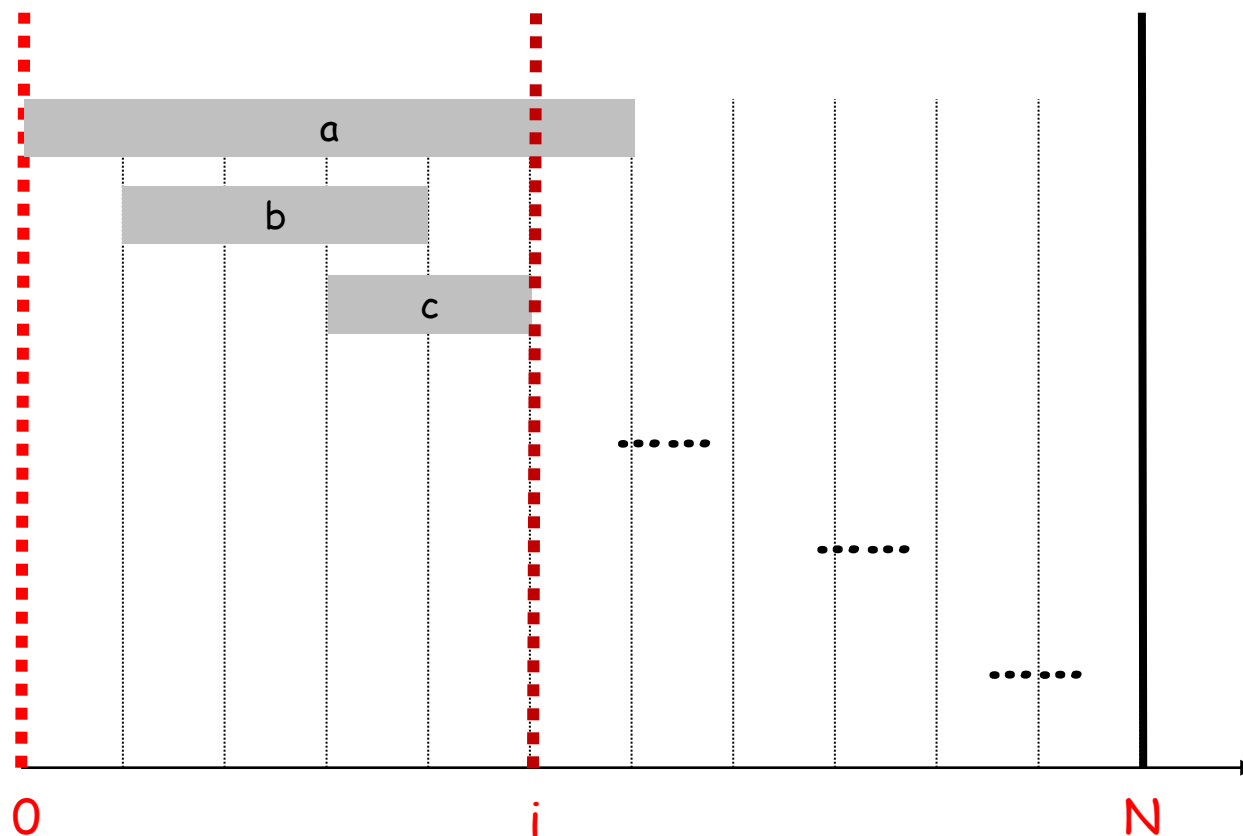


递归：问题规模缩小



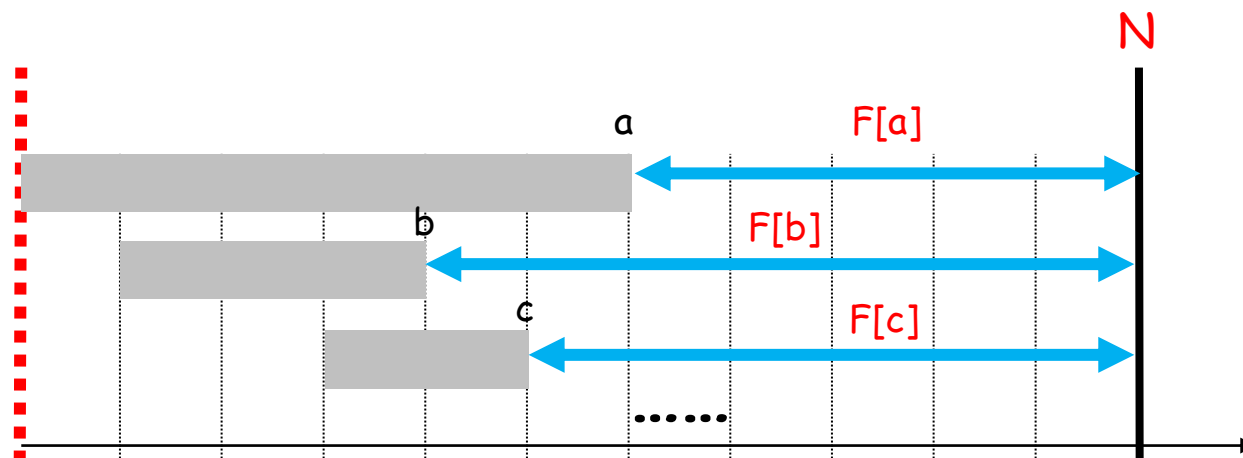
递归：问题规模缩小

如何缩小规模？尝试纳入一个具体的选择



对于 $F[i]$ 来说，枚举在它隶属范围内 $[i, N]$ 的（有可能成为最终解的）第一个选择

递归：问题规模缩小



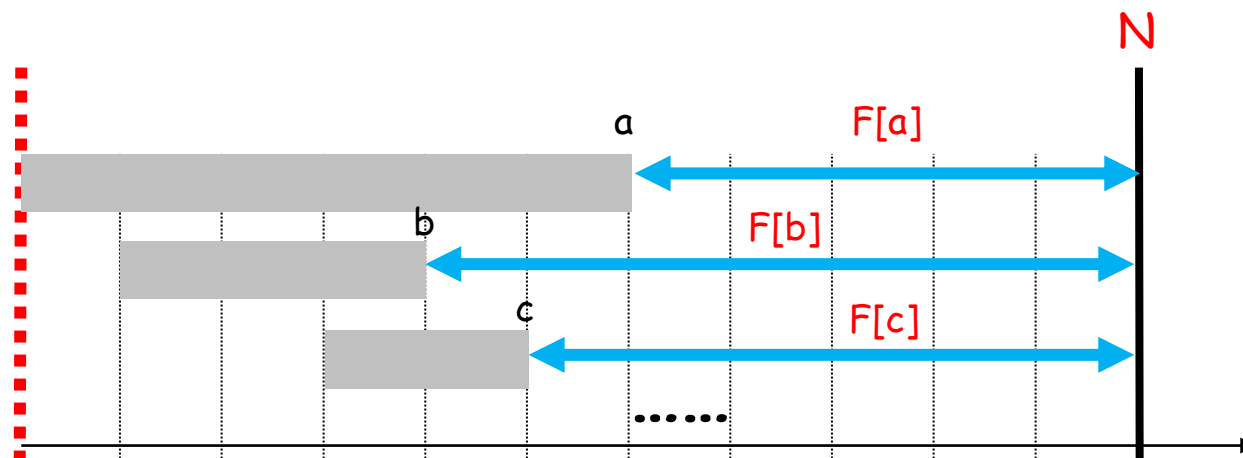
$F[0] = \max\{$
 $1 + F[a],$
 $1 + F[b],$
 $1 + F[c],$

}

$F[b] \geq F[c] \geq F[a]$

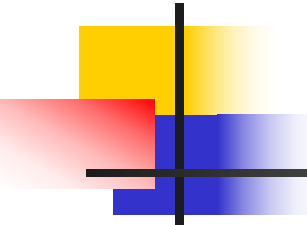
发现一个规律!

递归：问题规模缩小



$F[0] = 1 + F[b]$, b 为其隶属区间结束时间最早的一个区间

解毕。因此本题可用贪心法求解。



实现与运行时间

- 贪心算法. 把任务(需求)按照结束时间递增排序。依次选取与前面已选定任务**相容**的新任务.

算法时间复杂度? $N \log N$

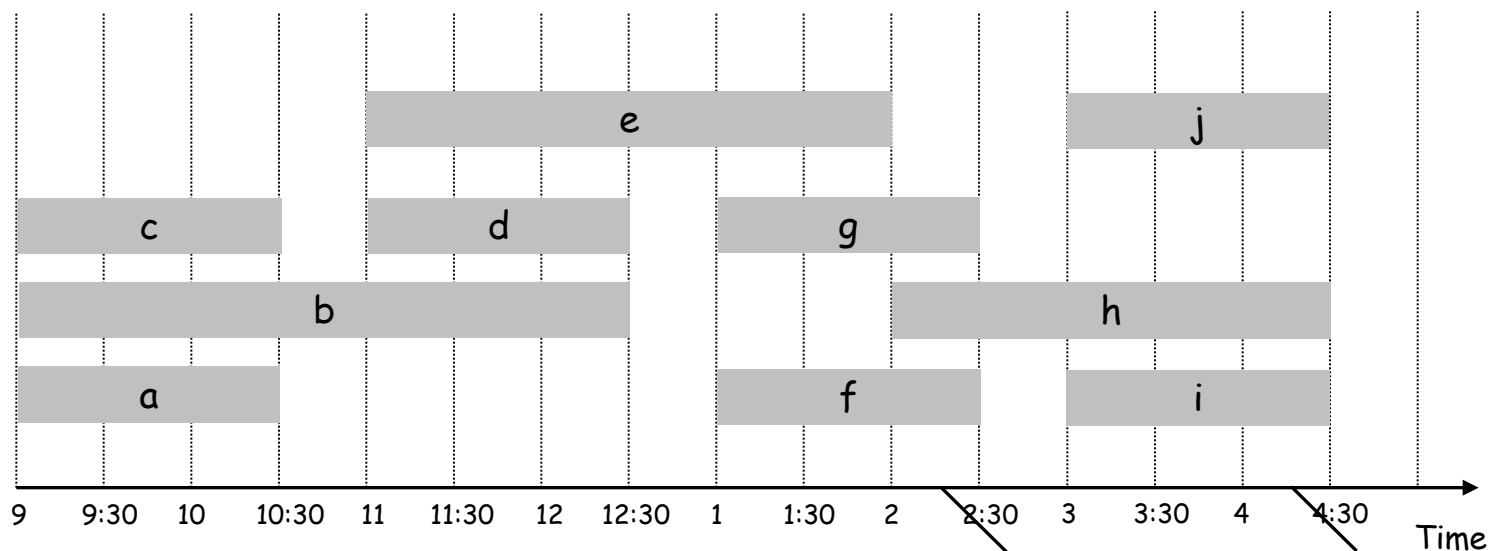


区间划分(Interval Partitioning)

- 区间调度问题中，存在一种单一的资源和时间区间表示的很多需求；如果我们拥有许多相同的资源可用，而且想尽可能用少的资源安排所有的需求，那么产生一个问题：**区间划分问题**。
- 应用场景：每个需求与特定时间区间教室里安排的课程(演讲)对应，我们想尽可能用少的教室满足所有这些需求。

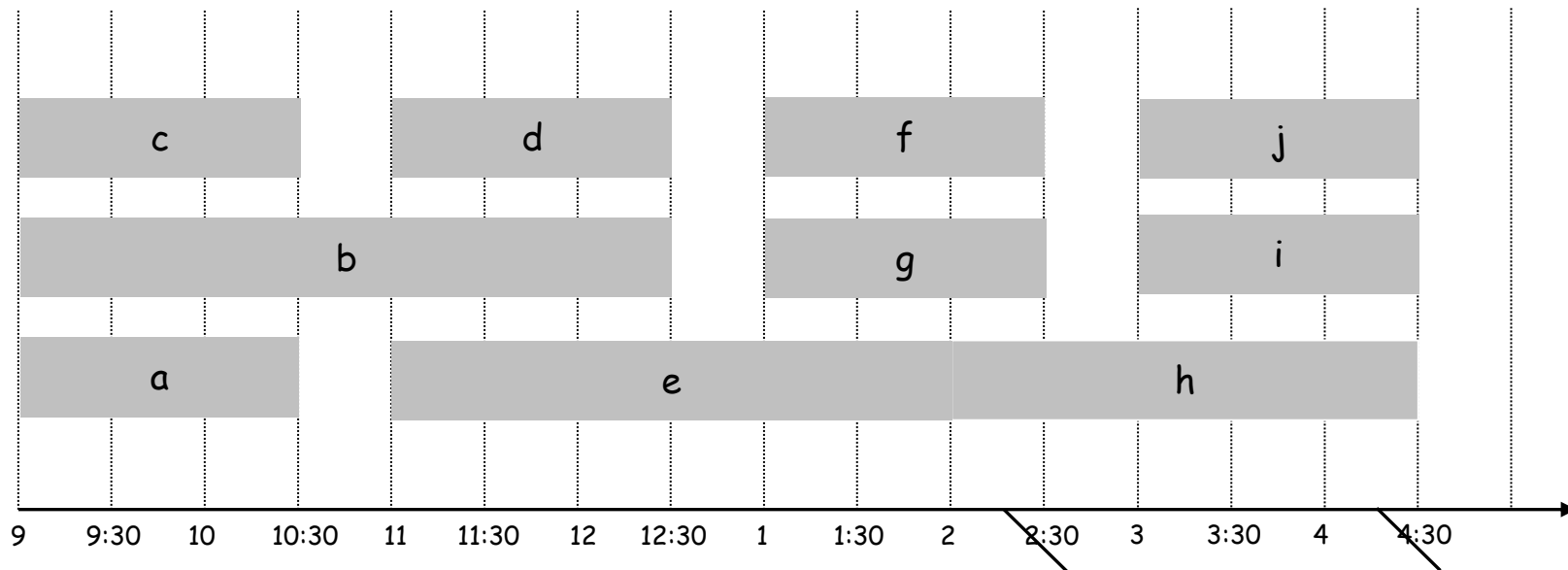
区间划分

- 区间划分问题描述.
 - 演讲 j 从 s_j 开始, 在 f_j 结束.
 - 目标: 寻找尽可能少的教室满足所有的这些演讲需求, 其中同一个教室中没有演讲冲突.
- Ex: 下面的计划用4个教室安排了10个演讲需求。



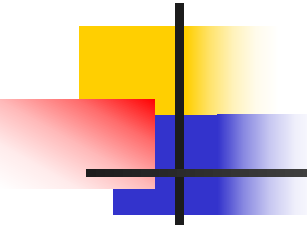
区间划分

- 有没有更好的方案？用更少的教室？



只需要3个教室!

Time

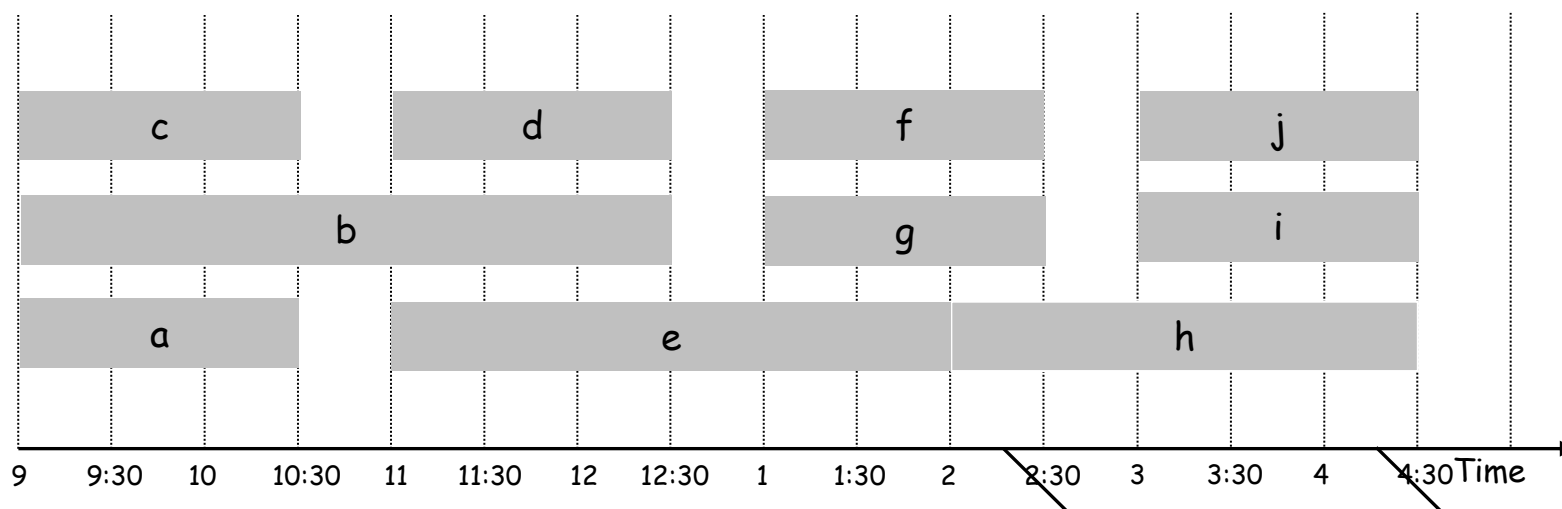


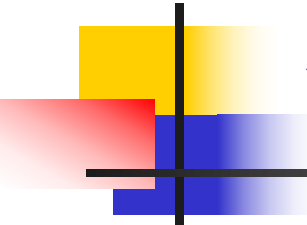
区间划分

- 观察一些关键的因素
- 定义：一个区间集合的深度是通过时间线上任何一点的最大区间数。
- 命题4.4 在任何区间划分的实例中，资源数必须至少是区间集合的深度。

区间划分

- **Ex:** 下面例子中，区间集合的深度 = 3 $\Rightarrow$ 所以给出的划分安排就是最佳方案.
- 问题: 是不是总存在一个区间划分方案使得需要的资源数等于区间集合的深度?





设计算法

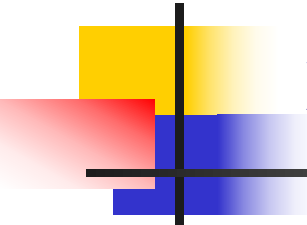
- 贪心算法要点：需求(演讲)按照开始时间排序，把需求安排到不冲突的教室中。

```
Sort intervals by starting time so that  $s_1 \leq s_2 \leq \dots \leq s_n$ .  
d  $\leftarrow$  0  
  
for j = 1 to n {  
    if (lecture j is compatible with some classroom k)  
        schedule lecture j in classroom k  
    else  
        allocate a new classroom d + 1  
        schedule lecture j in classroom d + 1  
        d  $\leftarrow$  d + 1  
}
```



算法分析

- 命题4.5 如果我们采用上面的贪心算法，每个区间将被分配一个标签，且没有两个重叠的区间接受同样的标签。
- 证明：
 - ✓ 没有两个重叠的区间分给同样的标签。
 - ✓ 不会有区间在结束时标签不够用；
考虑一个区间 I ，以及存在 t 个区间早于 I 且与 I 重叠，那么 $t+1 \leq d$ ，从而 d 个标签中至少还剩一个没有分配给 I 。



算法分析与实现

- 实现方法:
 - 对于每一个教室 k , 记录下最后一个需求的结束时间;
 - 把(已分配)教室放在一个优先队列中(按照结束时间先后)。
- 实现复杂度代价? $O(n \log n)$



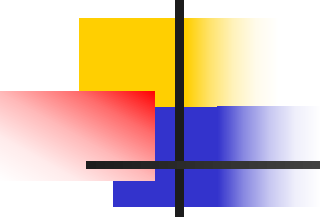
优先队列

优先队列的定义与实现方法

优先队列 (Priority Queue) 是一种抽象数据结构，其特点是每个元素都有一个优先级，出队时按优先级最高的元素先出队（而不是普通队列的先进先出）。它的核心操作是插入元素（按优先级排序）和取出优先级最高的元素。

优先队列的核心特性

1. 插入元素：将元素按优先级插入队列。
 2. 取出元素：取出当前队列中优先级最高的元素。
 3. 时间复杂度：
 - 理想情况下，插入和取出操作的时间复杂度应低于 $O(n)$ 。
 - 典型实现（如堆）的时间复杂度为 $O(\log n)$ 。
-



3. 二叉堆（最常用）

- 特点：完全二叉树，父节点的优先级总是高于（或低于）子节点。
 - 最大堆：父节点值 $\geq$ 子节点（用于取出最大值）。
 - 最小堆：父节点值 $\leq$ 子节点（用于取出最小值）。
- 时间复杂度：
 - 插入（上浮）： $O(\log n)$
 - 取出（下沉）： $O(\log n)$
- 空间复杂度： $O(n)$ ，通常用数组实现。

以下是**最小堆（Min Heap）**的 上浮（Sift Up） 和 下沉（Sift Down） 操作的可视化过程，结合文字分步解释和示意图：

一、最小堆的插入与上浮（Sift Up）

假设已存在一个最小堆，现在插入一个新元素 2：

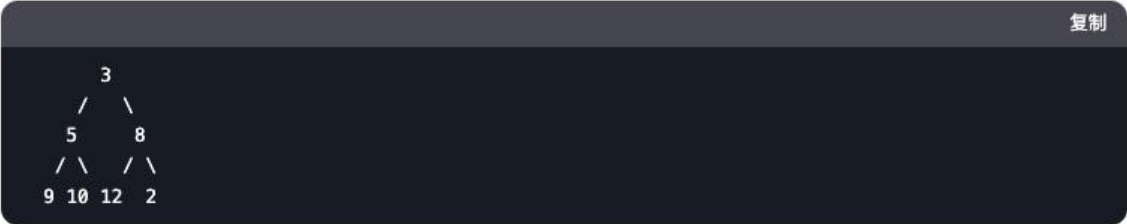
初始堆结构（插入前）



对应的数组： [3, 5, 8, 9, 10, 12]

步骤 1：插入元素 2

将 2 添加到堆的末尾（数组末尾），此时堆结构被暂时破坏：



对应的数组： [3, 5, 8, 9, 10, 12, 2]

步骤 2：上浮操作（从新元素 2 开始）

1. 比较父节点：新元素 2 的父节点是 8（索引 $(6-1)//2 = 2$ ）。
2. 交换条件：因为 $2 < 8$ ，交换它们的位置：



对应的数组： [3, 5, 2, 9, 10, 12, 8]

3. 继续比较父节点：新位置（索引 2）的父节点是 3（索引 $(2-1)//2 = 0$ ）。
4. 交换条件：因为 $2 < 3$ ，再次交换：



对应的数组： [2, 5, 3, 9, 10, 12, 8]

最终堆结构



二、最小堆的删除与下沉 (Sift Down)

假设从最小堆中删除根节点 2 (取出最小值):

初始堆结构 (删除前)



对应的数组: [2, 5, 3, 9, 10, 12, 8]

步骤 1: 删除根节点

- 1. 将根节点 2 取出。
- 2. 将堆的最后一个元素 8 移动到根节点:



对应的数组: [8, 5, 3, 9, 10, 12]

步骤 2: 下沉操作 (从根节点 8 开始)

- 1. 比较子节点: 根节点 8 的左子节点是 5, 右子节点是 3。
- 2. 选择更小的子节点: $3 < 5$, 所以选择右子节点 3。
- 3. 交换条件: 因为 $8 > 3$, 交换它们的位置:



对应的数组: [3, 5, 8, 9, 10, 12]

- 4. 继续比较子节点: 新位置 (索引 2) 的左子节点是 12, 没有右子节点。
- 5. 交换条件: 因为 $8 < 12$, 无需交换, 下沉结束。

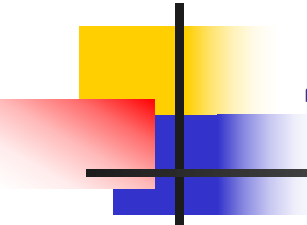
最终堆结构



示意图对比

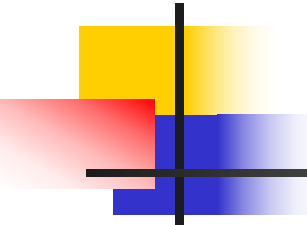


通过这种可视化，可以直观理解最小堆如何通过 **上浮** 和 **下沉** 维护其性质。



4.2 最小延迟调度

- 问题：我们有单一资源和一组使用资源的 n 个需求，每个需求需要一个时间区间。假定需求 i 更加灵活，不是开始时间与结束时间，需求 i 有一个截至时间 d_i ，要求一个长度为 t 的连续的时间区间。目标是需要尽可能多的满足需求。



最小延迟调度

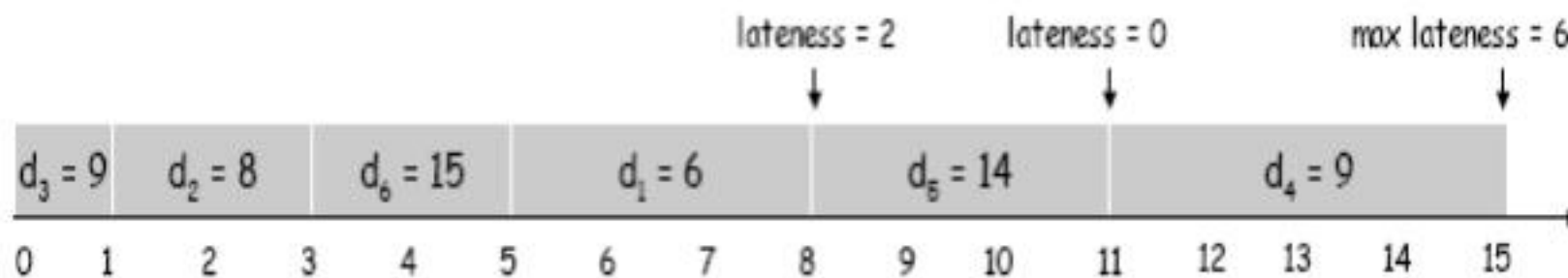
描述

- 需求 j 需要长度为 t_j 时间段, 截至时间为 d_j .
- 需求 j 如果在 s_j 开始, 那么结束时间是 $f_j = s_j + t_j$.
- 延迟的定义: $l_j = \max \{ 0, f_j - d_j \}$.
- 目标: 安排所有的需求, 使得计划具有最小的延迟调度:
让 $L = \max l_j$ 最小.

例子

Ex:

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



设计算法

- 方案3 根据直觉基础, 应该保证具有**最早截至时间**的任务完成的更早一些。
- 按照结束时间 d_i 增长的次序排序(最早截止时间优先)

Ex:

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15

求和

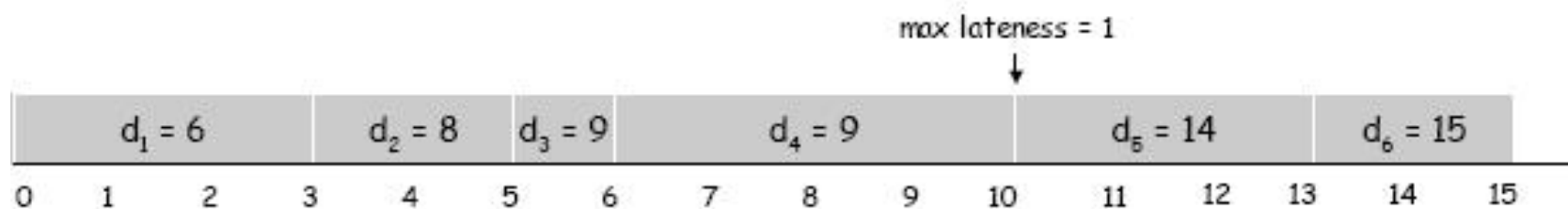
做差

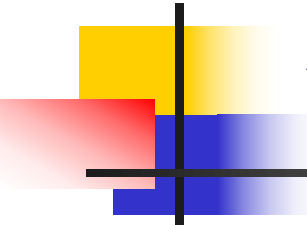
固定!!

设计算法

■ 例子

	1	2	3	4	5	6
t_i	3	2	1	4	3	2
d_i	6	8	9	9	14	15





设计算法

■ 贪心算法(最早截至时间优先)

```
Sort n jobs by deadline so that  $d_1 \leq d_2 \leq \dots \leq d_n$ 
```

```
t  $\leftarrow$  0
```

```
for j = 1 to n
```

```
    Assign job j to interval [t, t + tj]
```

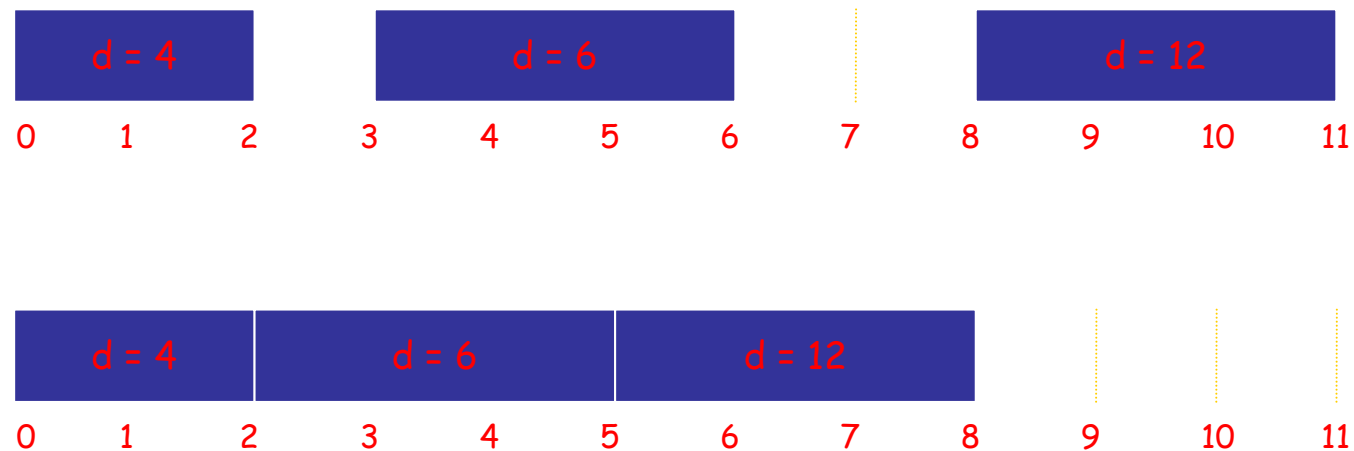
```
    sj  $\leftarrow$  t, fj  $\leftarrow$  t + tj
```

```
    t  $\leftarrow$  t + tj
```

```
output intervals [sj, fj]
```

分析算法

- 问题观察：不应有时间“空隙”存在



分析算法

- 尝试如下方式刻画调度：一个调度 A' 有一个**逆序**，如果具有截止时间 d_i 的任务 i 被安排在具有更早截止时间 $d_j < d_i$ 的任务 j 的前边。





分析算法

- 如果存在很多具有相等截至时间的任务，可能存在很多不同的没有逆序的调度，**对这些任务安排次序的不同会不会有影响？**

命题4.8 所有没有逆序也没有空闲时间的调度有相同的最大延迟。



分析算法

- 证明：两个不同的调度仅可能在具有相同截止时间的任务安排次序上不一样。这两个不同调度中，具有截止时间 d 的任务全部被连续安排。在这些任务中，**最后一个有着最大的延迟**，而且这个延迟不依赖于这些任务的次序。

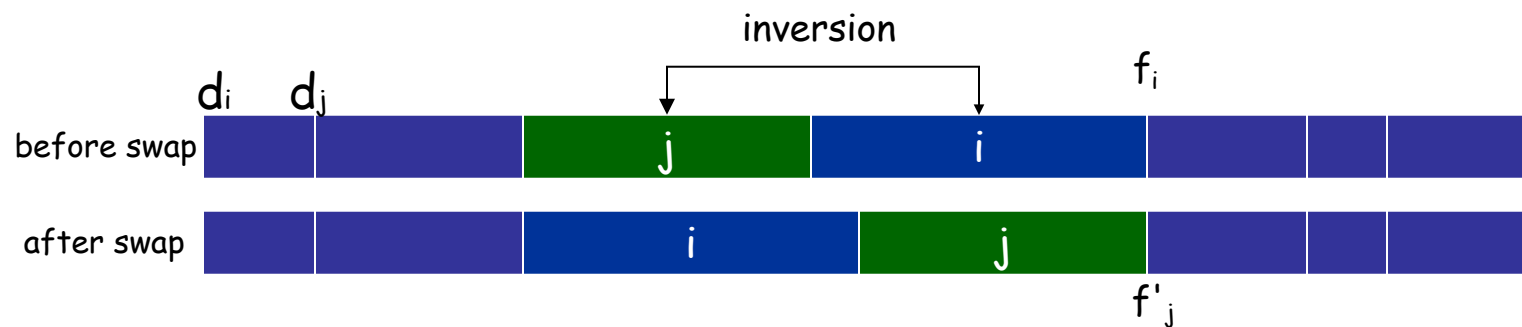


分析算法

- 先考虑最简单情形(没有空闲时间): 如果存在一个**相邻**的逆序任务调度, 那么交换以后, 所得到的调度方案是不是更好?
- 命题: 如果调度中存在一个**相邻**的逆序任务 i, j , 那么交换以后, 所得到的新调度方案不会增加最大延迟。

分析算法

■ 证明:



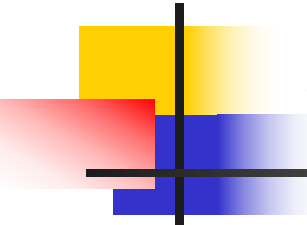
设 ℓ 是交换之前的延迟, ℓ' 是交换后的延迟。

■ $\ell'_k = \ell_k$ for all $k \neq i, j$

■ $\ell'_i \leq \ell_i$

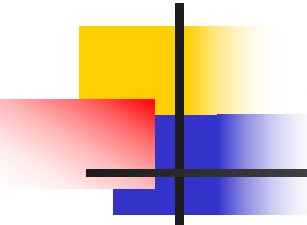
■ 那么

$$\begin{aligned}\ell'_j &= f'_j - d_j && \text{(definition)} \\ &= f_i - d_j && (j \text{ finishes at time } f_i) \\ &\leq f_i - d_i && (i < j) \\ &= \ell_i && \text{(definition)}\end{aligned}$$



分析算法

- 定理4.9 存在一个即没有逆序，也没有空闲时间的最优调度。
- 证明
 - (1)如果调度 O 有一个逆序，那么存在一对相邻的逆序 i, j .
 - (2)交换 i 和 j 之后，我们得到具有减少一个逆序的新调度。
 - (3)新的被交换的调度的最大延迟不大于 O 的最大延迟；
 - (4)消除所有逆序。



分析算法

- 定理4.10 前面贪心算法产生的调度有最优的最大延迟。
- 证明： 定理4.9证明存在一个没有逆序的最优调度； 又根据命题4.8， 所有没有逆序的最优调度有着**相同**的最大延迟。 所以贪心算法产生最优解。



贪心算法分析要点

- 贪心算法领先(Greedy algorithm stays ahead)
- 交换论证(Exchange argument)
- 结构论证(Structural) 发现一个所有解具有的界限("structural bound"), 然后严格证明算法能够达到这个界。



超高速缓存

- **超高速缓存** 一个快速存储器中存储少量数据以便减少与一个慢速存储器的交互而花费的时间
- 比如:
 - ✓ 主存对于硬盘像超高速缓存
 - ✓ 硬盘对于因特网像超高速缓存
 - ✓ 你的办公桌对于校图书馆像超高速缓存



超高速缓存

- 目的是为了**让超高速缓存尽可能有效**，当你访问某块数据时，数据应该尽可能在超高速缓存里
- 需要一个**超高速缓存维护算法**，确定数据何时存在超高速缓存里，何时从超高速缓存中收回。



超高速缓存

- 问题抽象:
- 主存中有 n 块数据的集合 U
- 更快的超高速缓存, 一次能够保存 $k < n$ 个数据。从 U 中取出一系列数据项: $D = d_1, d_2, \dots, d_n$; 需要决定在每个时刻哪 k 个项保存在超高速缓存中。



超高速缓存

- 对某项 d_i ,如果已经在超高速缓存中(访问命中, **cache hit**), 那么访问速度很快; 否则, 需要从主存中调配过来。这时, 需要收回在超高速缓存中别的数据, 为 d_i 空出位置。这种情形叫做**超高速缓存缺失(cache miss)**。我们希望尽可能少出现这些缺失, 少做交换。



超高速缓存

- 对于一个特定的存储访问序列，一个超高速缓存算法决定一个**收回调度**。在哪些点把哪些项从超高速缓存中收回，这样就确定了超高速缓存的内容和随时间的交换数目。

超高速缓存

- Ex: $k = 2$, 初始 cache = ab,
访问序列: a, b, c, b, c, a, a, b.

a	a	b
b	a	b
c	c	b
b	c	b
c	c	b
a	a	b
a	a	b
b	a	b
requests	cache	

Cache交换数目?

2



超高速缓存

- 系统研究人员关注的
- 给定一个存储访问的完全序列，什么是使得超高速缓存交换尽可能少的收回调度？



超高速缓存

- “最远将来规则” (Farthest-in-future)

When d_i 需要被放入超高速缓存

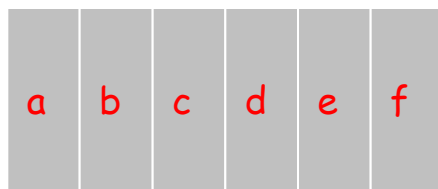
收回在**最远的将来**被需要的那个项

- 日常生活中的一些经验

超高速缓存

■ 例子

current cache:



future queries:

g a b c e d a b b a c d e a f a d e f g h ...

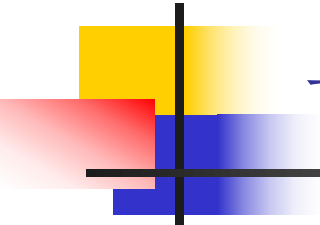
↑
cache miss

↑
eject this one



超高速缓存

- Theorem. [Bellady, 1960s] FF (Farthest-in-future) 调度方案是最优调度方案，得到最小的交换次数。

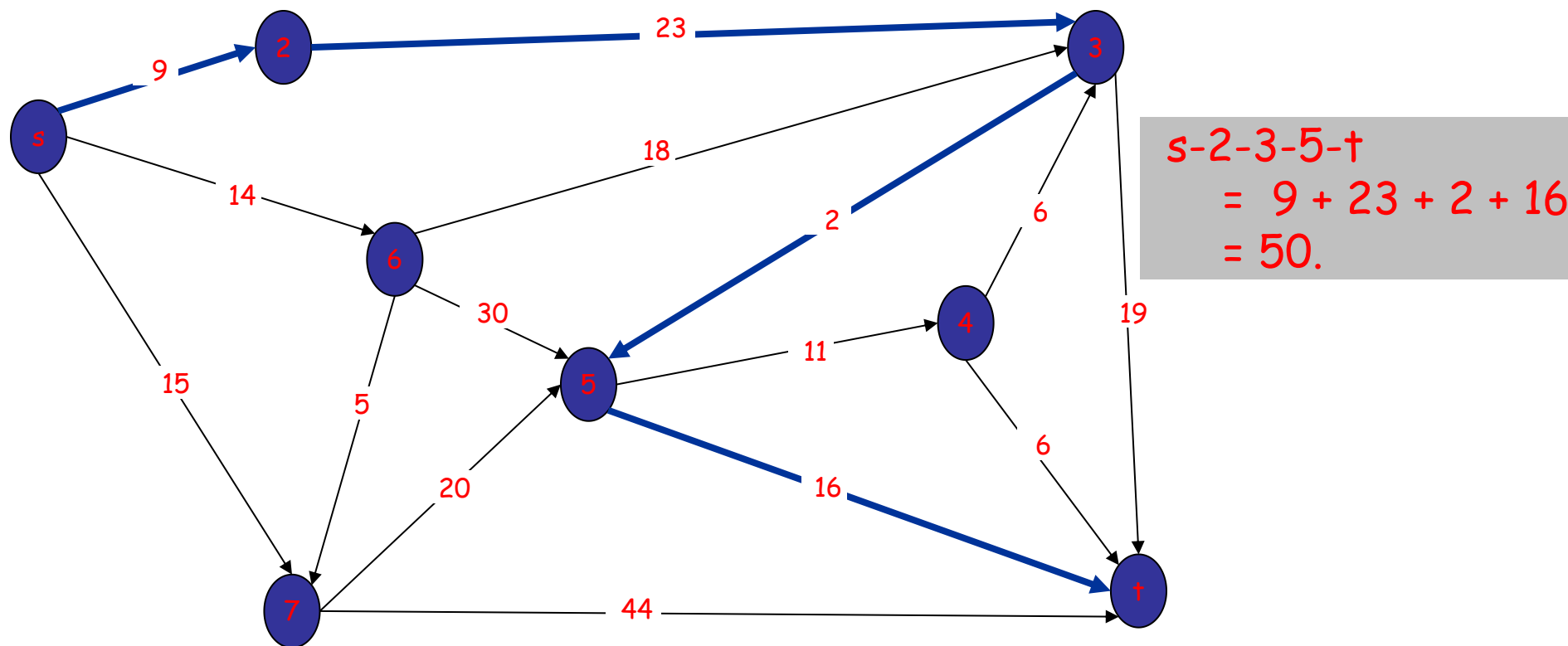


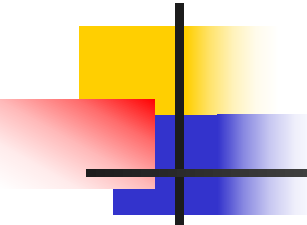
一个图的最短路径

- Shortest path network
 - Directed graph $G = (V, E)$.
 - Source s , destination t .
 - Length ℓ_e = length of edge e .
- 最短路径问题: 寻找有向图中 s 到 t 的最短路径

一个图的最短路径

最短路径长度是多少？





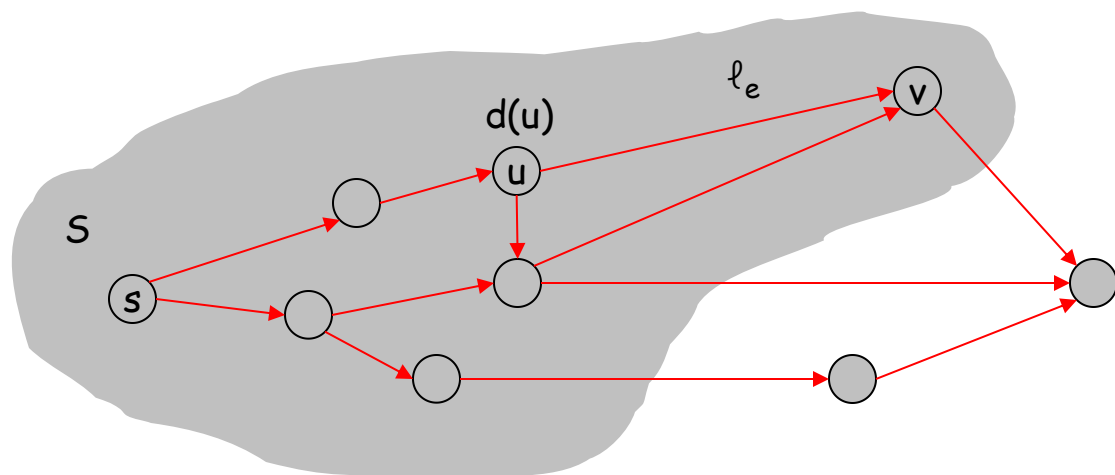
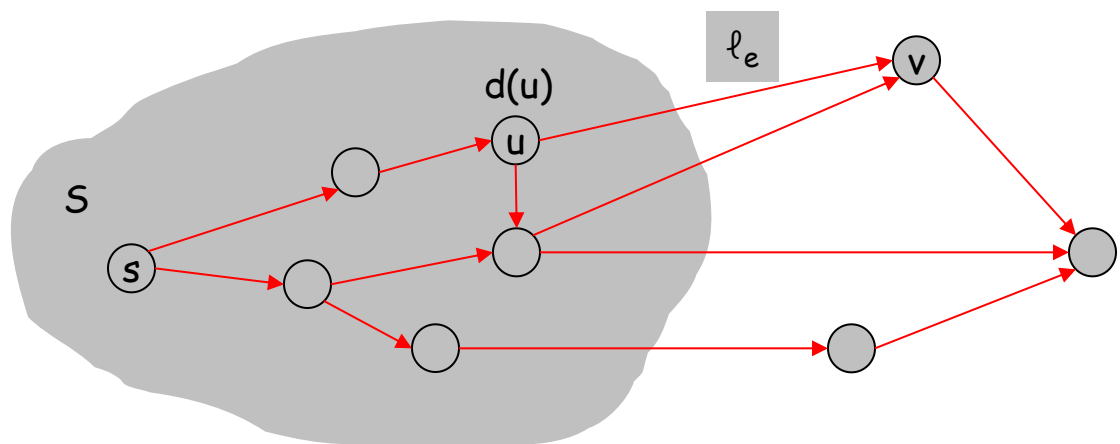
Dijkstra算法

- 保持一个结点集 S ，如果源点 s 到 u 的最短路径一旦确定，那么 u 就加到 S 中。
- 最开始 $S = \{s\}$, $d(s) = 0$.
- 循环寻找结点 v ，使得 $\pi(v) = \min_{e=(u,v): u \in S} d(u) + \ell_e$,

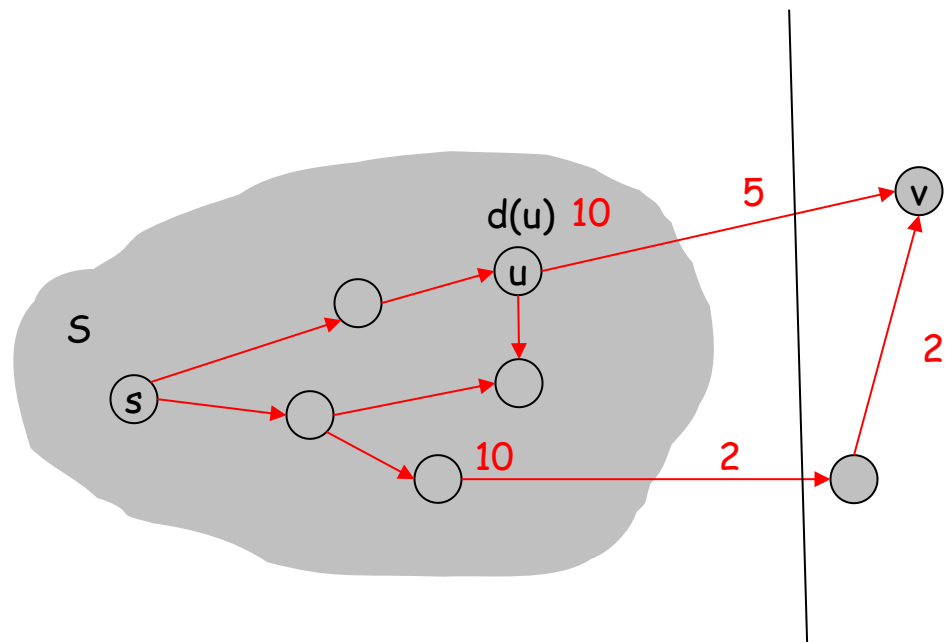
从已经探索过的某个结点 u 到 v 的边 (u, v)

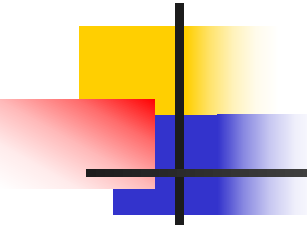
最小，然后把 v 加入 S ，并设 $d(v) = \pi(v)$.

Dijkstra算法



Dijkstra算法





Dijkstra算法

Dijkstra算法(G, l)

设 S 是被探查的结点集合

对每个 S 中的 u , 存储一个 $d(u)$;

初始 $S=\{s\}$ 且 $d(s)=0$

While $S \neq V$

选择一个结点不在 S 中的结点 v , 使得从 S 到 v 至少有一条边连接并且

$d'(v) = \min_{e=(u,v), u \in S} d(u) + l_e$ 最小

将 v 加入 S 并且定义 $d(v) = d'(v)$

Endwhile

分析算法

- Pf. 采用归纳法
- $|S| = 1$ 易证.
- 假设 $|S| = k \geq 1$ 命题成立。
 - Let v be next node added to S , and let $u-v$ be the chosen edge.
 - The **shortest** $s-u$ path plus (u, v) is an $s-v$ path of length $\pi(v)$.
 - **Consider any $s-v$ path P . We'll see that it's no shorter than $\pi(v)$.**
 - Let $x-y$ be the first edge in P that leaves S , and let P' be the subpath to x .
 - P is already too long as soon as it leaves S .

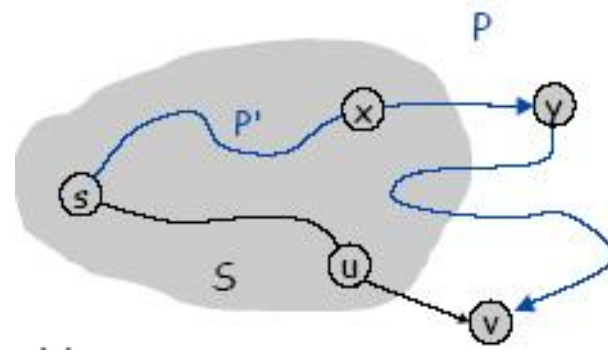
$$\ell(P) \geq \ell(P') + \ell(x, y) \geq d(x) + \ell(x, y) \geq \pi(y) \geq \pi(v)$$

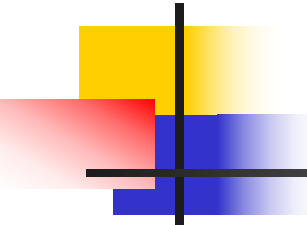
↑
nonnegative
weights

↑
inductive
hypothesis

↑
defn of $\pi(y)$

↑
**Dijkstra chose v
instead of y**





算法实现

- 对于每个“无关顶点”，不做改变
- 对于“相邻顶点”，当 v 被加入“核心”时，对于边 $e = (v, w)$ ，更新临时最短距离

$$\pi(w) = \min \{ \pi(w), \pi(v) + l_e \}.$$

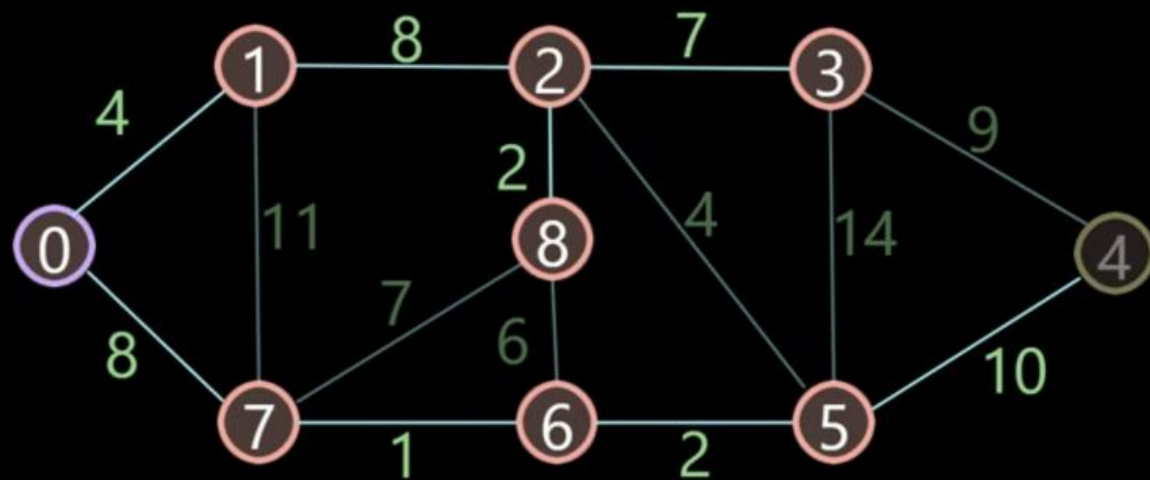
- 提高效率的方法：采用优先队列来存放没有探索的结点，根据 $\pi(v)$ 来排序。



算法分析

- 普通实现代价
每次考虑增加一个结点;
求到源点最小距离.
复杂度: $O(mn)$
- 采用优先队列实现代价?
 $O(m)$ 次考虑边, n 次ExtractMin
 m 次ChangeKey;
复杂度: $O(m\log n)$

1. 每次从未标记的节点中选择距离出发点最近的节点，标记，收录到最优路径集合中。
2. 计算刚加入节点A的邻近节点B的距离（不包含标记的节点），
若（节点A的距离+节点A到节点B的边长）< 节点B的距离，就更新节点B的距离和前面点。



	出发点0	前面点
0 ✓	0	
1 ✓	4	0
2 ✓	12	1
3 ✓	19	2
4	21	5
5 ✓	11	6
6 ✓	9	7
7 ✓	8	0
8 ✓	14	2



4.5 最小生成树问题

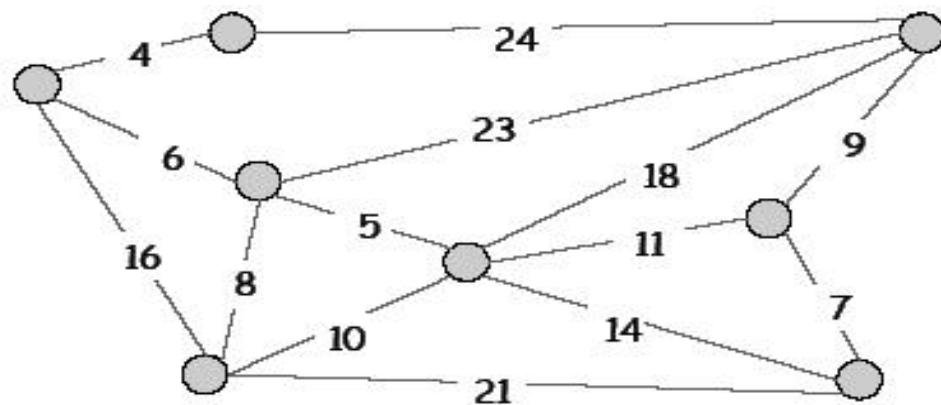
- 假设我们有位置集合 $V = \{v_1, v_2, \dots, v_n\}$, 我们想建立一个通信网络, 网络应该是连通的, 我们希望尽可能便宜的建立它。

- 问题抽象:

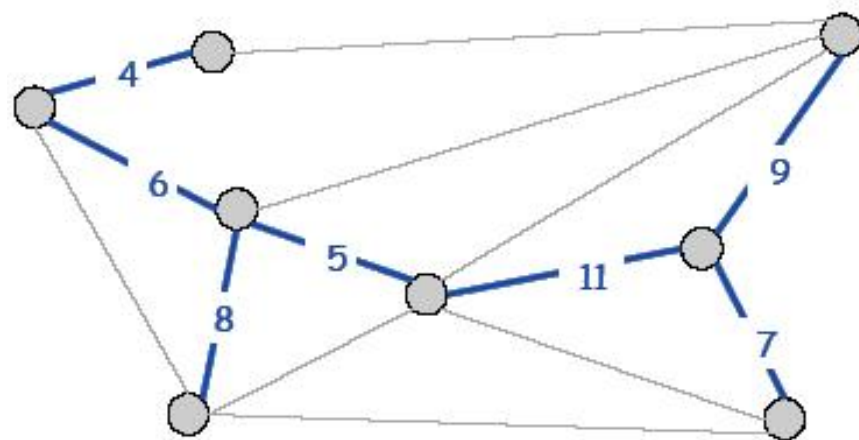
对于确定的边 $e = (v_i, v_j)$, 存在一个正的费用 C_e , 问题是对于图 $G = (V, E)$, 寻找 $T \subseteq E$

使得图 (V, T) 是连通的, 而且 $\sum_{e \in T} C_e$ 最小。

最小生成树问题



$G = (V, E)$



$T, \sum_{e \in T} c_e = 50$



贪心算法设计

- **Kruskal's algorithm.** 初始 $T = \phi$. 边按照费用递增次序排列。通过不断插入边来建立一棵生成树: 把边 e 插入 T 只要不构成圈。
- **Reverse-Delete algorithm(反向删除算法).** 初始 $T = E$. 依照费用递减的次序开始删除边。当达到每条边 e 时(从最贵的边开始), 只要这样做不破坏当前图的连通性。



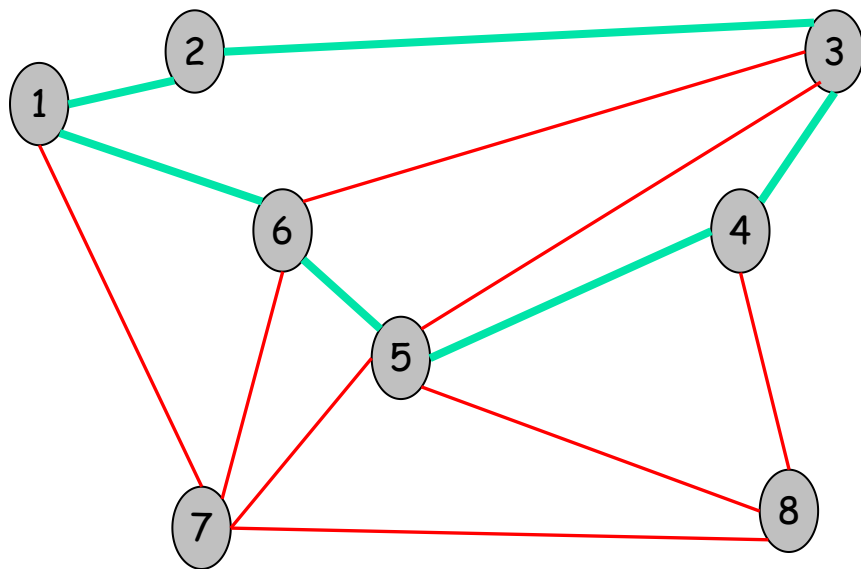
贪心算法设计

- Prim's algorithm. 初始 $S=\{s\}$, 然后贪心选择的生长树 T . 每步选择一端在 T 中, 费用最小的边与 T 连接。

殊途同归, 三种算法都可以产生最小生成树(MST)!

圈和割

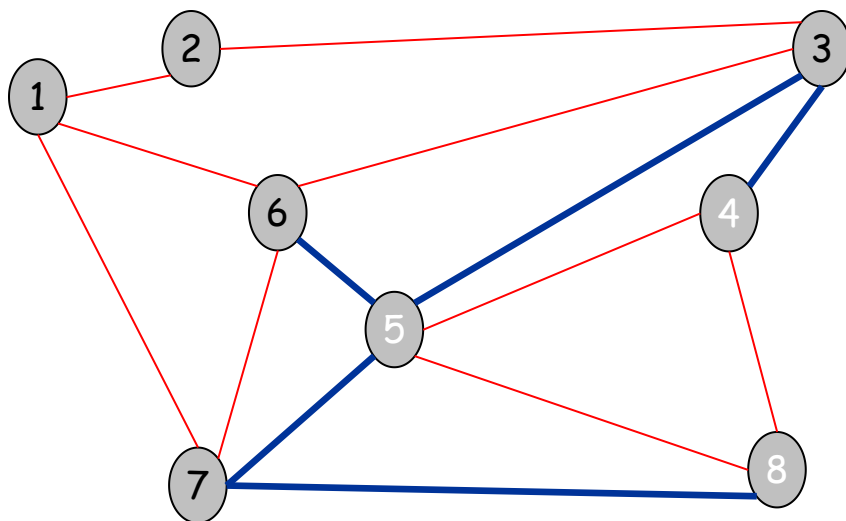
- 圈(Cycle). Set of edges the form $a-b, b-c, c-d, \dots, y-z, z-a$.



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$

圈和割

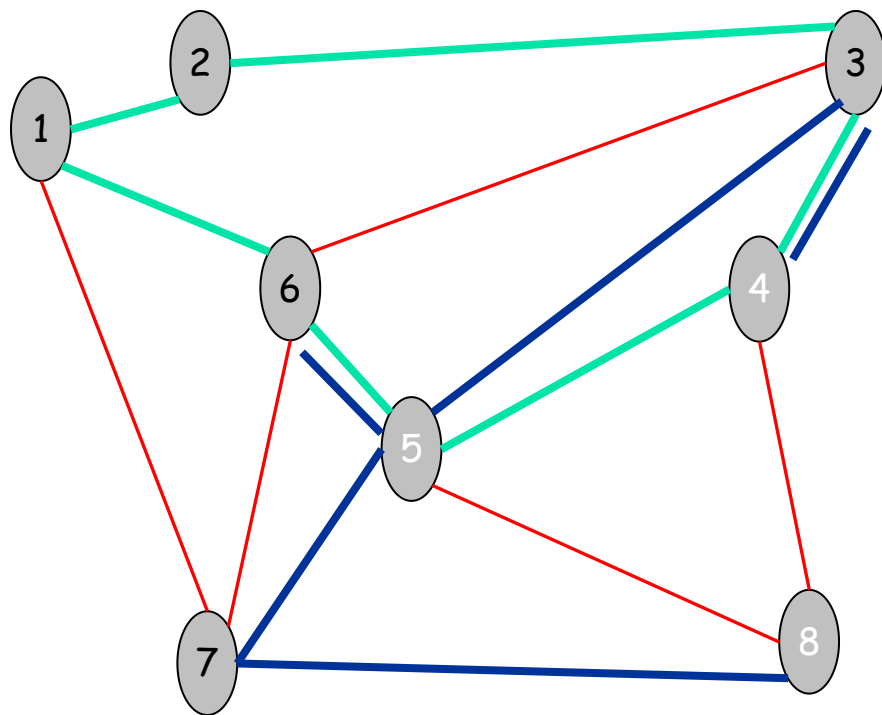
- 割 (Cutset) A cut is a subset of nodes S . The corresponding cutset D is the subset of edges with exactly one endpoint in S .



Cut S = $\{4, 5, 8\}$
Cutset D = $5-6, 5-7, 3-4, 3-5, 7-8$

圈和割

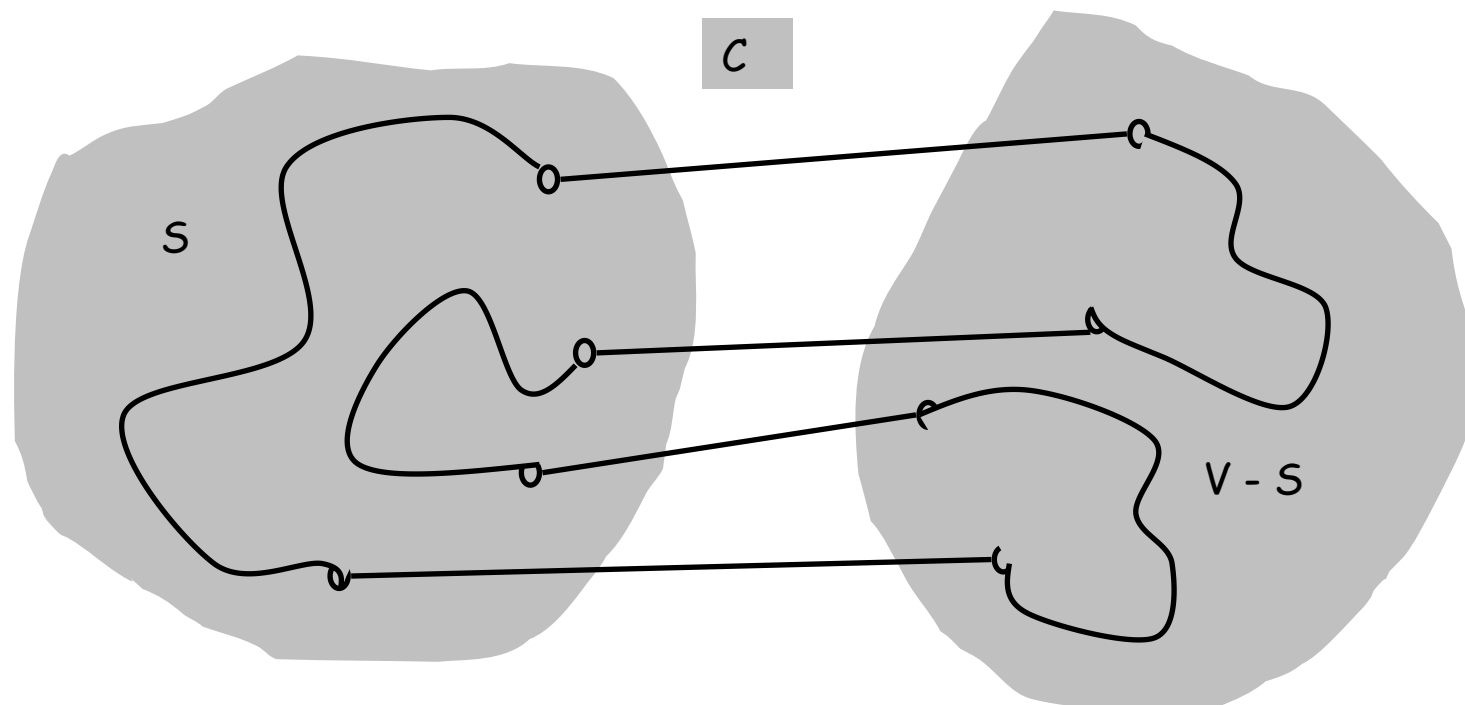
- 命题：一个圈和一个割有偶数条相交边。



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$
Cutset $D = 3-4, 3-5, 5-6, 5-7, 7-8$
Intersection = 3-4, 5-6

圈和割

■ 证明:

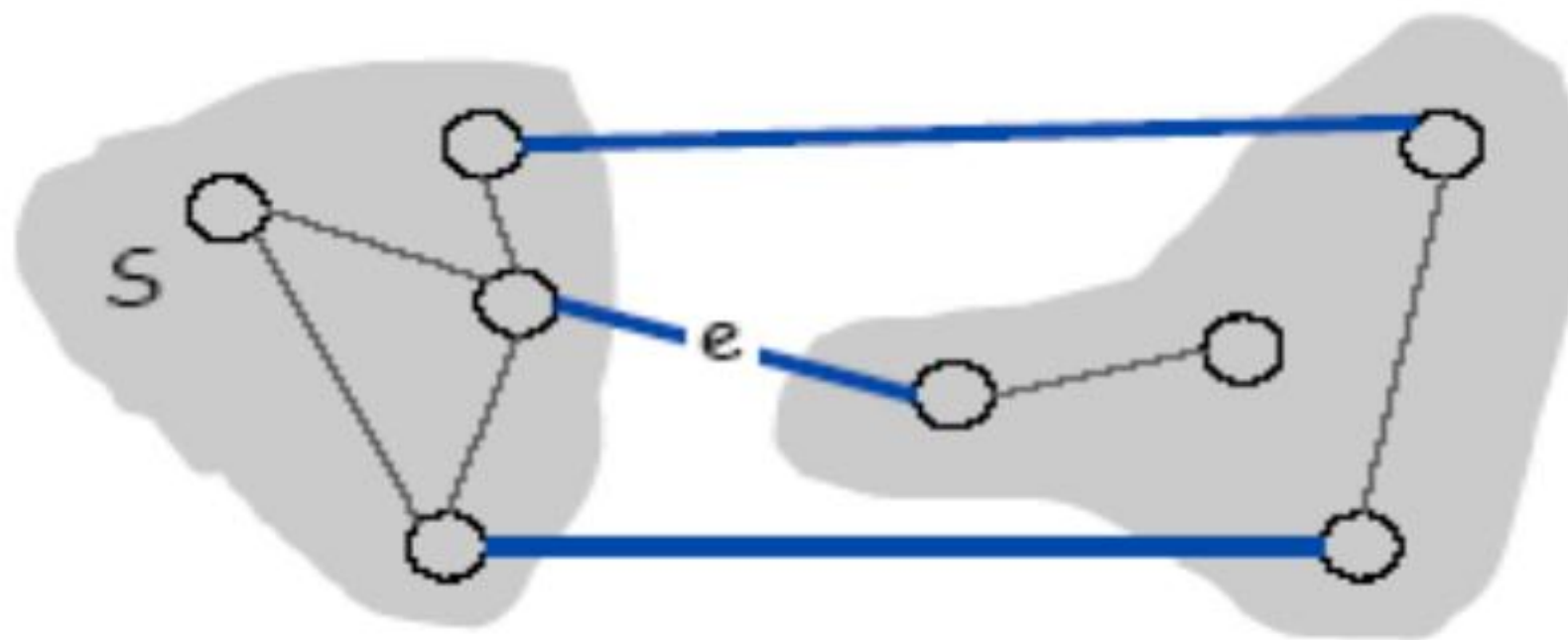




算法分析

- 什么时候在最小生成树中包含一条边是安全的？
- 定理4.17 假设所有边的费用都是不等的。令 S 是任意节点子集, $S \neq \emptyset, V$, 令边 $e=(u,v)$ 是一端在 S 中, 另一端在 $V-S$ 中的最小费用边。那么每棵最小生成树都包含边 e .

算法分析



e is in the MST

算法分析

■ Pf. (采用交换论证)

假设 **e** 不属于 **T***(最小生成树).

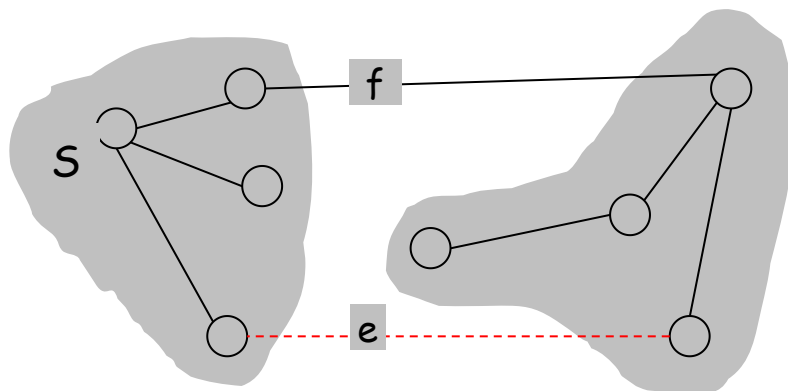
增加 **e** 到 **T*** 形成了一个圈**C**.

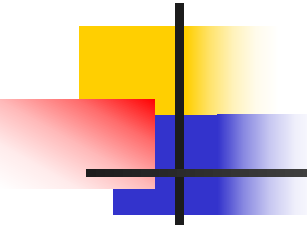
边**e** 即属于圈 **C**, 也属于与**S**对应的割 **D**; $\Rightarrow$ 一定存在另一条边**f**, 即属于**C**也属于 **D**.

$T' = T^* \cup \{e\} - \{f\}$ 也是一个生成树.

因为 $c_e < c_f$, $\text{cost}(T') < \text{cost}(T^*)$.

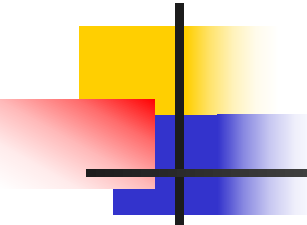
导出矛盾.





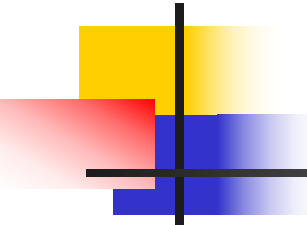
Prim 算法

- Prim's algorithm. [Jarník 1930, Dijkstra 1957, Prim 1959]
 - 初始 $S =$ 任意结点.
 - 每次把与 S 对应的割中最小费用边加入 T , 同时把新的探索结点 u 加入 S .



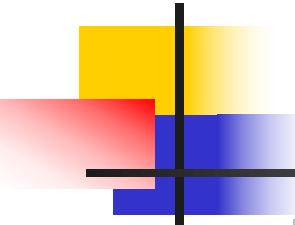
Prim算法

- 定理4.19 Prim算法产生一棵最小生成树。
- 证明： 对于Prim算法，容易知道它只加了那些属于每棵最小生成树的边。根据定义，**e是使得一个结点在S中而另一个结点在V-S中费用最低的边**($\min_{e=(u,v), u \in S} C_e$)，所以根据割性质4.17，e在每一棵最小生成树中。



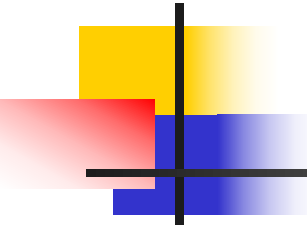
Prim算法

- 实现. 采用与Dijkstra 类似的算法, 使用优先队列.
 - 保持一些已经探索过的结点集合S.
 - 对于每一个没有探索过的结点v, $\text{cost } a[v] = \text{cost of cheapest edge } v \text{ to a node in } S$.
 - 复杂度估计: 采用优先队列 (堆), $O(m \log n)$.



Prim算法

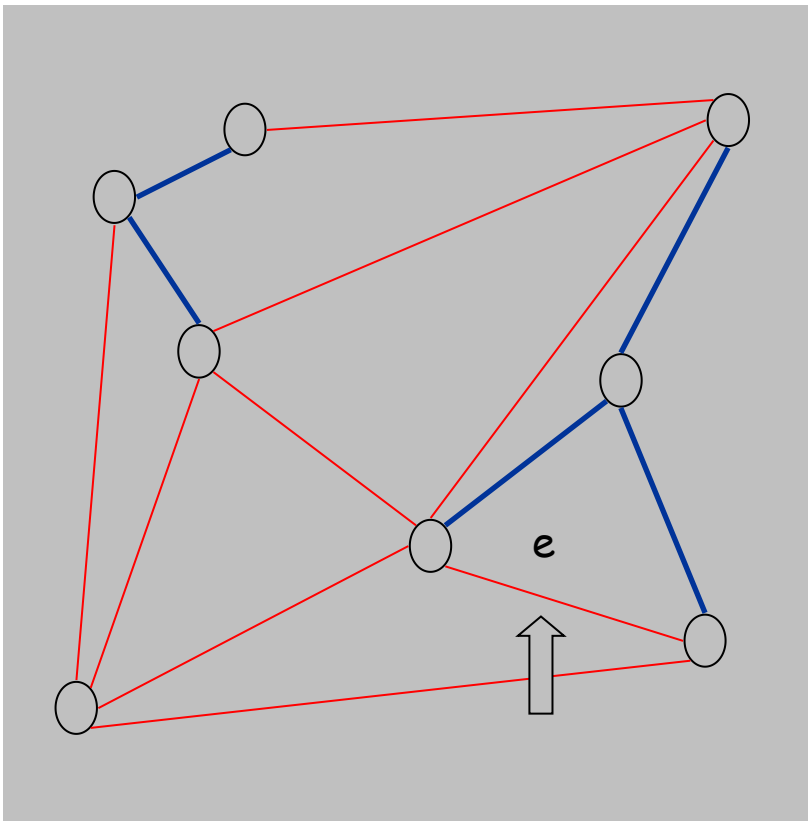
```
Prim(G, c) {  
    foreach (v ∈ V) a[v] ← ∞  
    Initialize an empty priority queue Q  
    foreach (v ∈ V) insert v onto Q  
    Initialize set of explored nodes S ← ∅  
  
    while (Q is not empty) {  
        u ← delete min element from Q  
        S ← S ∪ { u }  
        foreach (edge e = (u, v) incident to u)  
            if ((v ∉ S) and (ce < a[v]))  
                decrease priority a[v] to ce  
    }  
}
```



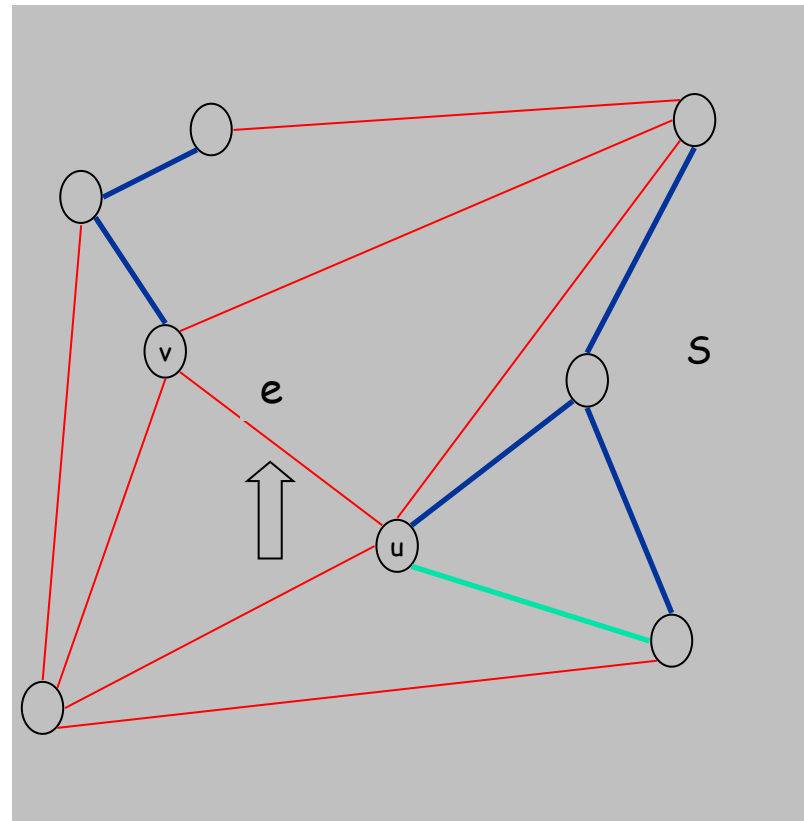
Kruskal算法

- Kruskal's algorithm. [Kruskal, 1956]
 - 边按照费用递增的顺序排序.
 - Case 1: 如果把边 e 加入 T 时构成了一个圈, 那么跳过边 e .
 - Case 2: 否则, 把边 $e = (u, v)$ 加入 T 中(根据割性质), 其中 S 是与 u 连通的结点集合.

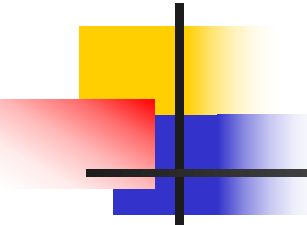
Kruskal算法



Case 1



Case 2



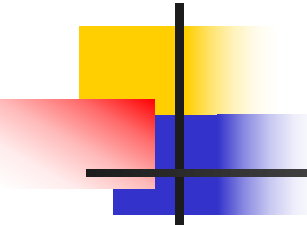
Kruskal算法

■ 算法实现

<https://zhuanlan.zhihu.com/p/93647900>

采用 **union-find** 数据结构.

- 生成一个最小生成树T.
- 保持每一个连通分支的集合.
- 算法代价: $O(m \log n)$.



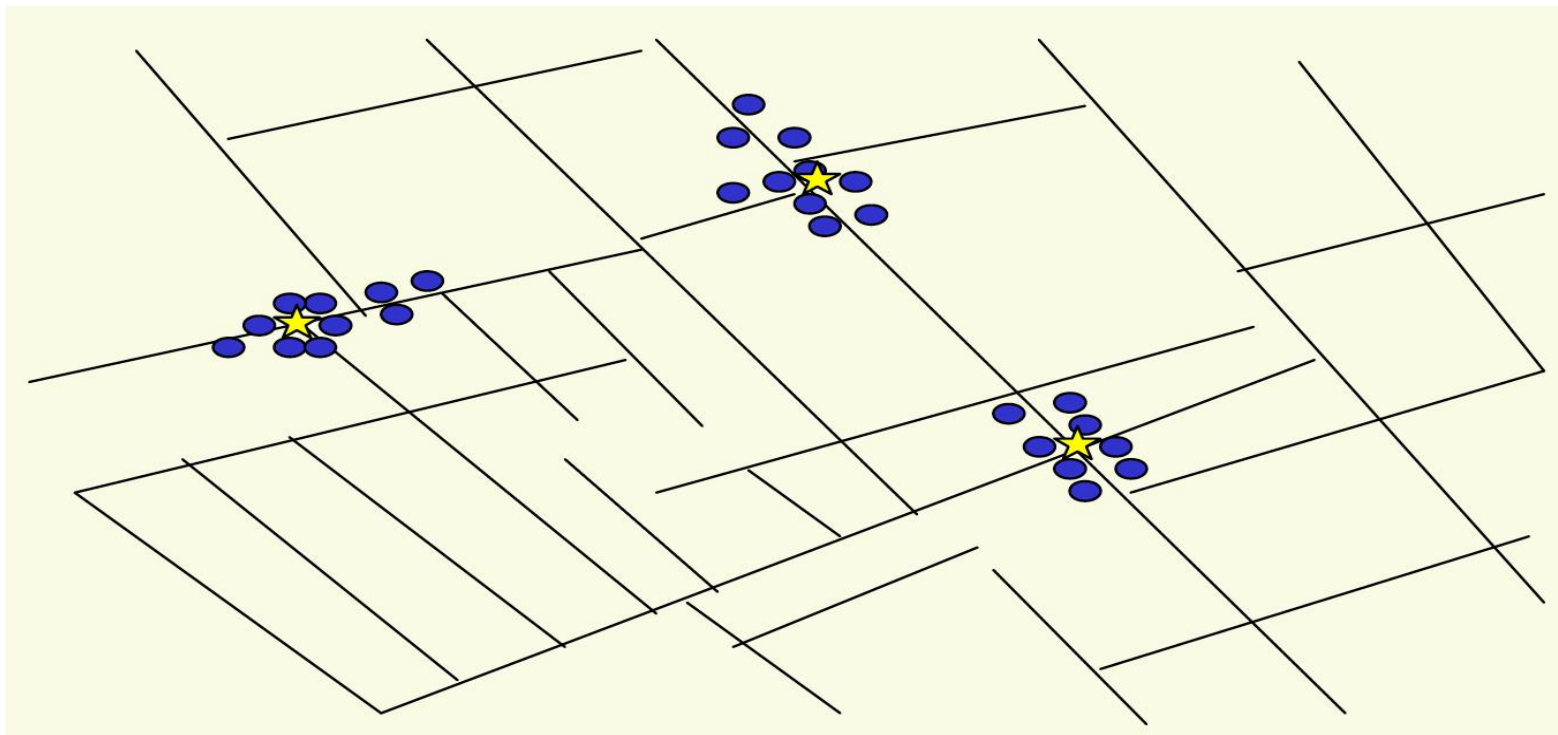
Kruskal算法

```
Kruskal(G, c) {  
    Sort edges weights so that  $c_1 \leq c_2 \leq \dots \leq c_m$ .  
     $T \leftarrow \phi$   
  
    foreach ( $u \in V$ ) make a set containing singleton u  
  
    for i = 1 to m  
        ( $u, v$ ) =  $e_i$   
        if (u and v are in different sets) {  
             $T \leftarrow T \cup \{e_i\}$   
            merge the sets containing u and v  
        }  
    return T  
}
```

排除所有边的费用不同的假设

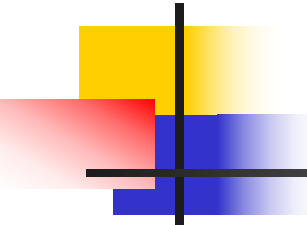
- 原来假设所有边的费用不同，现在推广：
- 对于边费用相同的情形加一个 *微小的扰动*，这样避免了“平分”的情形；对于原来不同的费用次序没有影响。注意到相关的算法基于边的费用的**相对关系**，所以同样产生对原始实例也是最优的**MST**.

聚类



Outbreak of cholera deaths in London in 1850s.
Reference: Nina Mishra, HP Labs

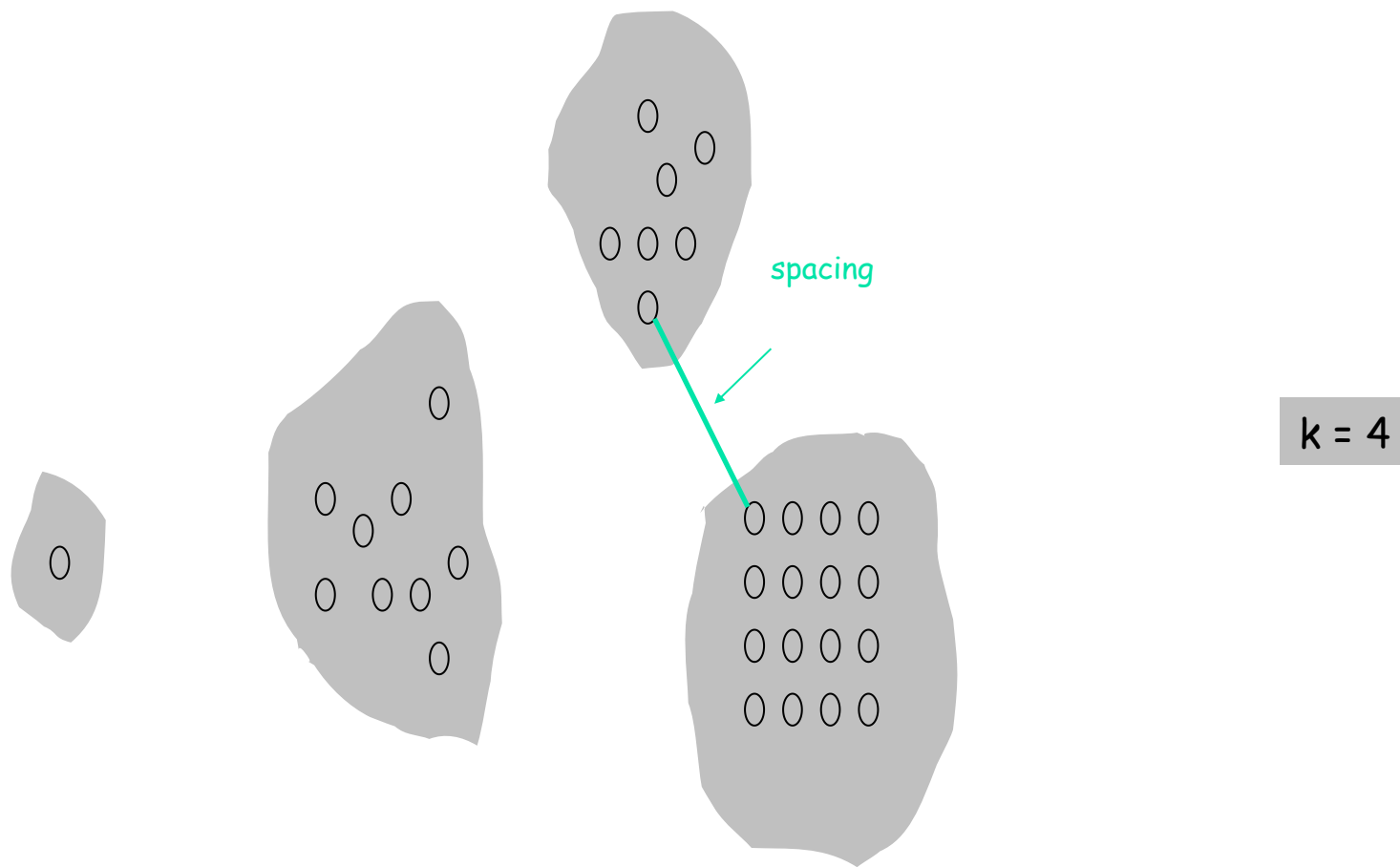
1850年代伦敦的霍乱疫情是公共卫生史上的重要事件，尤其是约翰·斯诺（John Snow）通过地图分析发现霍乱传播与受污染水源有关，成为流行病学研究的里程碑。



聚类

- 最大间隔聚类
- K聚类. 把对象分成k个非空的部分.
- 距离函数:
 - $d(p_i, p_j) = 0$ iff $p_i = p_j$
 - $d(p_i, p_j) \geq 0$
 - $d(p_i, p_j) = d(p_j, p_i)$
- 间隔. 处在不同聚类中的任何一对点之间的最小距离
- 对于某个k, 寻找具有**最大**可能间隔的 k聚类.

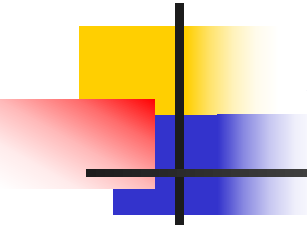
聚类





设计算法

- 可以发现，实现的过程仿佛就是Kruskal最小生成树算法
- 也就是，运行Kruskal算法，但是就在它加最后的 $k-1$ 条边之前停止。
- 等价于取整棵最小生成树 T ，然后删除 $k-1$ 条最贵的边，并且定义 k 聚类是所得到的连通分支 $C_1, C_2, \dots, C_k$.

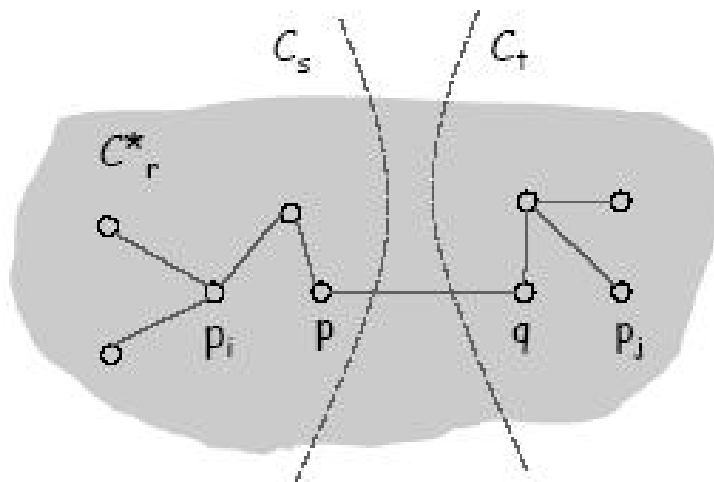


分析算法

- 定理4.26 由删除最小生成树T的k-1条最贵的边所构成的连通分支
组成一个最大间隔的k聚类。 $C^*: C_1^*, C_2^*, \dots, C_k^*$

分析算法

- 证明： 设 C : $C_1, \dots, C_k$ 是另外一个聚类
 - C^* 的间隔就是第 $k-1$ 条最贵边的边，长度为 d^* .
 - 设 p_i, p_j 在 C^* 的同一个聚类 C_r^* 中，但是却在 C 不同的聚类中，比如 C_s 与 C_t .
 - 由于 p_i, p_j 属于同一个连通分支 C_r^* ，所以一定在我们停止之前 Kruskal 算法添加了 p_i-p_j 路径上所有的边。特别的，这里 p_i-p_j 路径上每条边的长度至多是 d^* 。
 - 因为相邻的 p, q 属于聚类中不同的集合，所以 C 的间隔 $\leq d^*$ ■

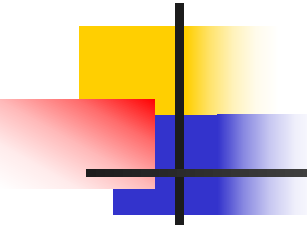


4.8 Huffman码与数据压缩

- 数据压缩，就是用最少的数码来表示信源所发出的信号，减少容纳给定消息集合或数据采样集合的信号空间
- 数字通信的基础

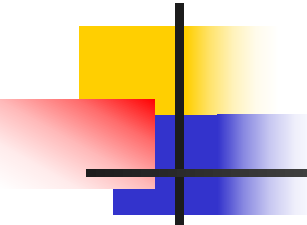
D. A. Huffman





问题的提出

- 数据压缩领域的基本问题之一:
- 字母用二进制表示, 字母的频率不同, 如何选择编码的方式, 使得效率最高?



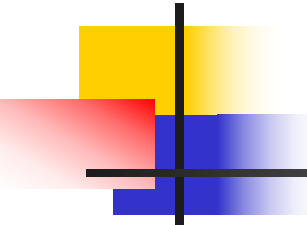
问题的提出

前缀码

- 对于一组字母**S**的前缀码是把每个字母 $x \in S$ 以下述方式映射到一个**0, 1**序列的函数 γ , 对于不同的 $x, y \in S$, 序列 $\gamma(x)$ 不是序列 $\gamma(y)$ 的前缀。

比如 $S = \{a, b, c, d, e\}$, 定义如下的编码方式

$$\gamma_1(a) = 11, \gamma_1(b) = 01, \gamma_1(c) = 001, \gamma_1(d) = 10, \gamma_1(e) = 000$$



问题的提出

- 最优前缀码

- 假设对每一个字母 $x \in S$, 文本中等于字母 x 的字母频率为 f_x , 字母总量为 n 。

那么编码总的长度为: $\sum_{x \in S} n f_x |\gamma(x)| = n \sum_{x \in S} f_x |\gamma(x)|$

我们的目标使得每个字母的平均位数

$$ABL(\gamma) = \sum_{x \in S} f_x |\gamma(x)| \text{ 最小。}$$



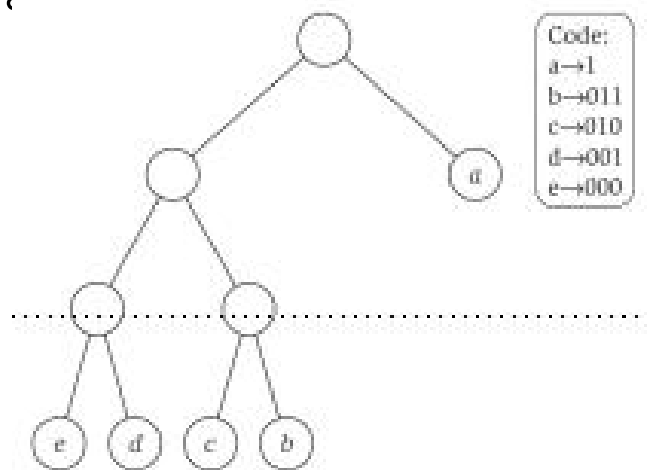
问题的提出

- 1948 Claude Shannon 奠定了编码的信息论基础
- 宏观的结论:
- 如果 γ 是一种可译编码方案(Instantaneous code), 那么 $ABL(\gamma) \geq H_r(f_1, f_2, \dots)$

其中
$$H_r(f_1, f_2, \dots) = \sum_{x \in S} f_x \log_r \frac{1}{f_x}$$

设计算法

- 使用**二叉树T**表示前缀码，那么叶结点就是我们需要的编码方式。



$$f_a = 0.32, f_b = 0.25, f_c = 0.20, f_d = 0.18, f_e = 0.05$$

- 可以知道，从**T**构成的**S**的编码是一个前缀码



设计算法

- 存在一个与树 T^* 对应的最优前缀码，其中两个最低频率的字母被指定为树叶，这两片树叶是 T^* 中的兄弟。
- 据此我们有了自底向上的算法策略：设 y^* 与 z^* 是最低频率的两个字母，用其父亲节点“元字母” w 替代 y^*, z^* (这样字母表缩小)，递归的寻找前缀码。



Figure 4.17 There is an optimal solution in which the two lowest-frequency letters label sibling leaves; deleting them and labeling their parent with a new letter having the combined frequency yields an instance with a smaller alphabet.



设计算法

■ 具体的算法步骤(Huffman算法):

To construct a prefix code for an alphabet S , with given frequencies:

 If S has two letters then

 Encode one letter using 0 and the other letter using 1

 Else

 Let y^* and z^* be the two lowest-frequency letters

 Form a new alphabet S' by deleting y^* and z^* and

 replacing them with a new letter w of frequency $f_{y^*} + f_{z^*}$

 Recursively construct a prefix code γ' for S' , with tree T'

 Define a prefix code for S as follows:

 Start with T'

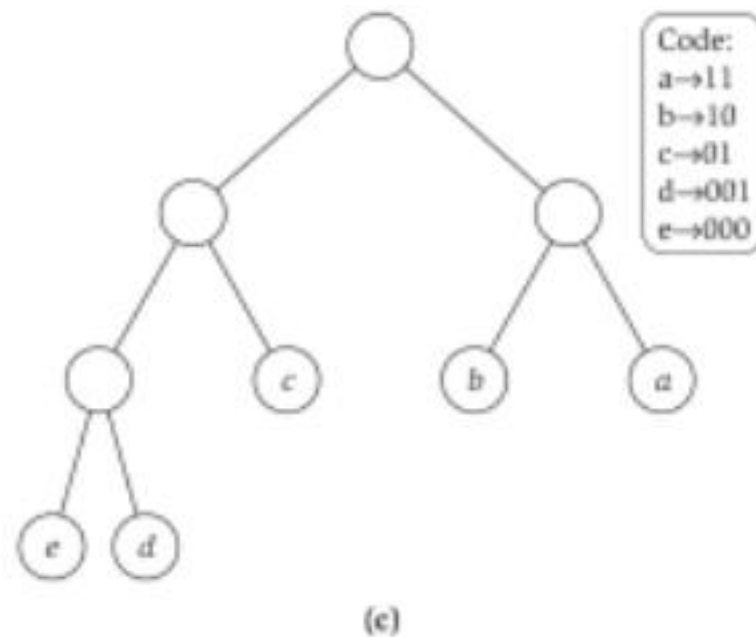
 Take the leaf labeled w and add two children below it
 labeled y^* and z^*

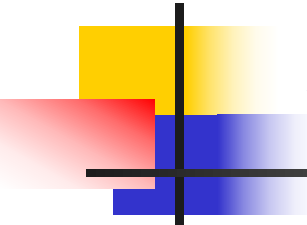
 Endif

设计算法

- $S=\{a,b,c,d,e\}$,
 - Step1 d,e 合并
 - Step2 c, (de) 合并
 - Step3 a,b 合并
 - Step4 (ab), (c,(de))合并
- 这样就得到了图(c)

$$f_a = 0.32, f_b = 0.25, f_c = 0.20, f_d = 0.18, f_e = 0.05$$





分析算法

实现与运行时间

- 一般的代价

识别最低频率的字母在 $O(k)$ 时间内,

迭代求和总的代价为 $O(k^2)$

- 采用优先队列(用堆实现)

每次插入和最小元素的取出调整时间 $O(\log k)$,

总运行时间 $O(k \log k)$